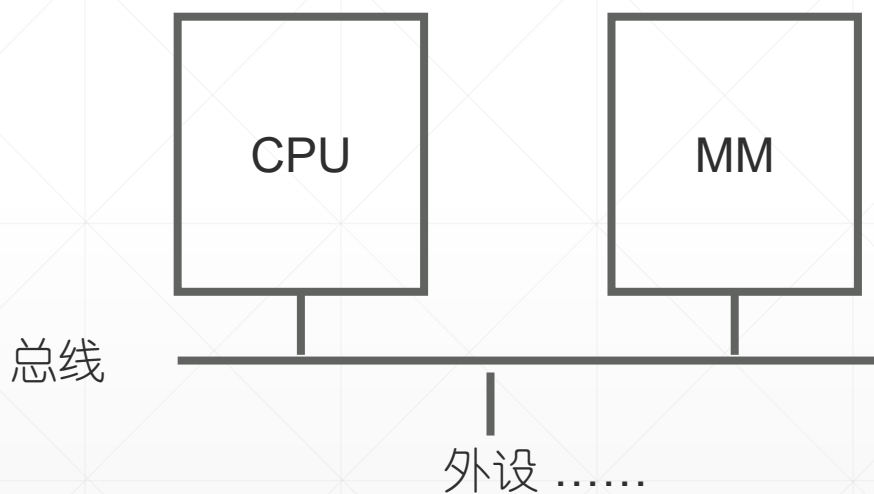


操作系统课程设计

第一部分、存储管理（物理内存管理）

同济大学计算机系

一、冯、诺伊曼架构计算机



1、存储程序思想

- 把程序存进内存
- CPU顺序执行内存中的程序，从入口开始，直至运行结束

2、内存（物理内存）是一个大数组

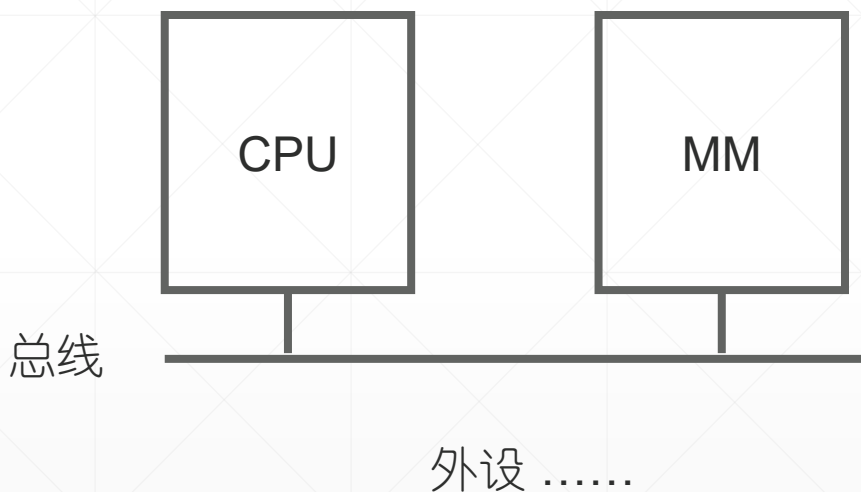
- 每个单元，1字节，8bit
- 数组下标，物理地址（PA，Physical Address）
- 数组长度，内存容量



3、内存中有什么

- 指令 和 数据
- 单条指令占据连续内存单元，最低字节地址是这条指令的地址
- 数据，同上

CPU访存过程



1、读操作

- CPU将PA放物理总线
- CPU设置控制总线，读操作
- 物理内存读出PA单元保存的字节，放数据总线
- CPU读数据总线，得到内存单元的值

2、写操作

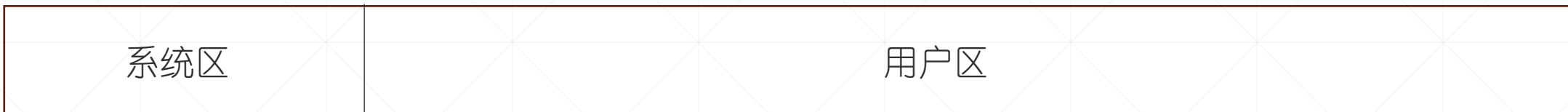
- CPU将PA放物理总线
- CPU将写入内存单元的值放数据总线
- CPU设置控制总线，写操作
- 物理内存读数据总线，将其写入PA标识的内存单元

3、内存访问速度，是系统性能瓶颈（内存墙）

- 内存访问速度 $\sim 200\text{ns}$
- CPU运算速度 $\sim 0.2\text{ns}$
- 每执行一条指令，至少访存一次。指令执行耗时 $\sim 200\text{ns}$

单道系统的内存，只有一个程序

0



系统区，存放OS内核。指令、数据从0编址。

用户区，存放正在执行的应用程序。指令、数据从M编址（M是用户区首字节地址）。

OS内核：若干供应用程序调用的内核函数。负责驱动外设，管理时钟，保障系统正常运行。
应用程序需要系统服务时，调用OS内核函数。

因为内存只有一个程序，所以指令和数据可以用物理地址。

多道系统的内存，存放多个进程图像



核心区，存放OS内核
用户区，按需分配给进程图像

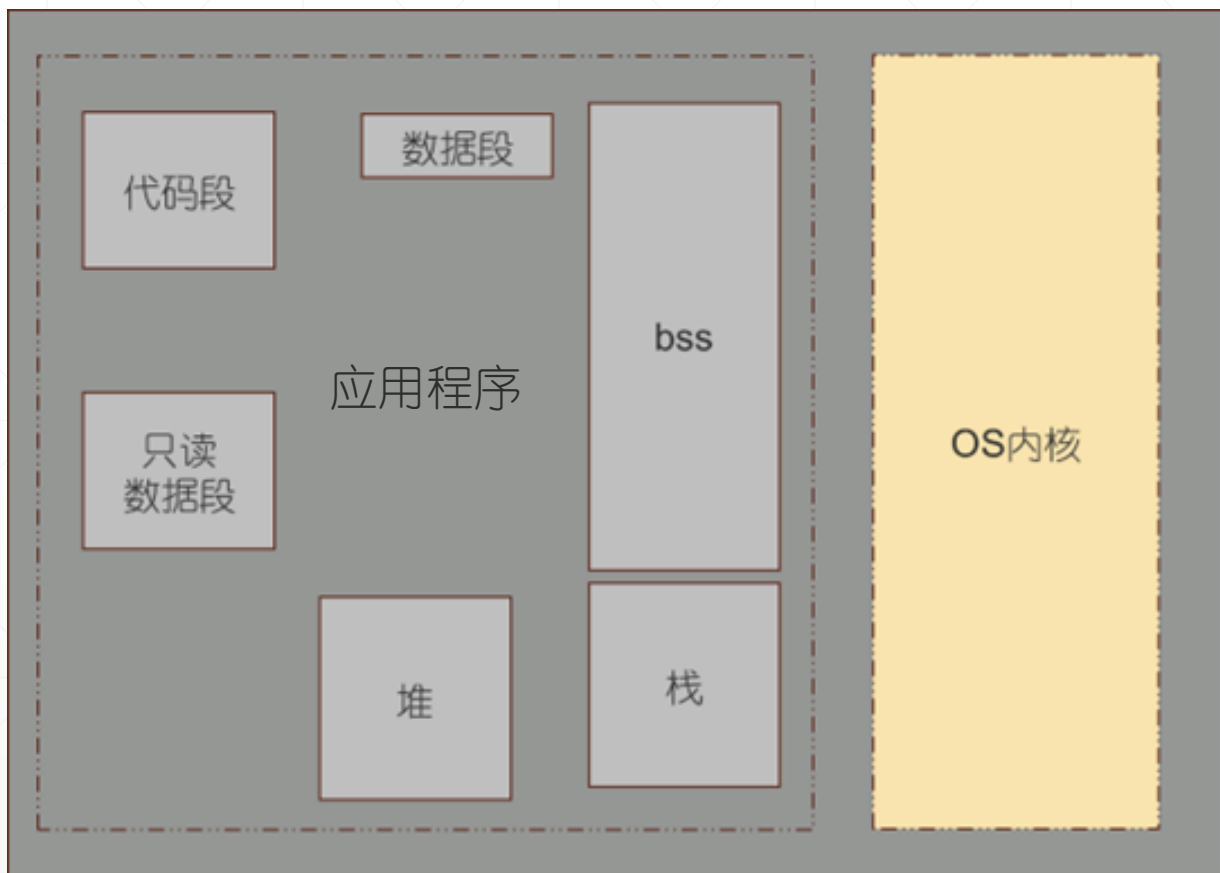
存储管理的任务

- 内存分配
- 地址映射
- 内存保护
- 内存共享
- 内存扩展

存储管理的目标

- 隔离，为进程创建封闭的运行环境，避免其它进程的干扰。
- 提高物理内存利用率，接纳更多进程图像（提高系统并发度）
- 加速指令执行，提高计算机运行速度

进程图像

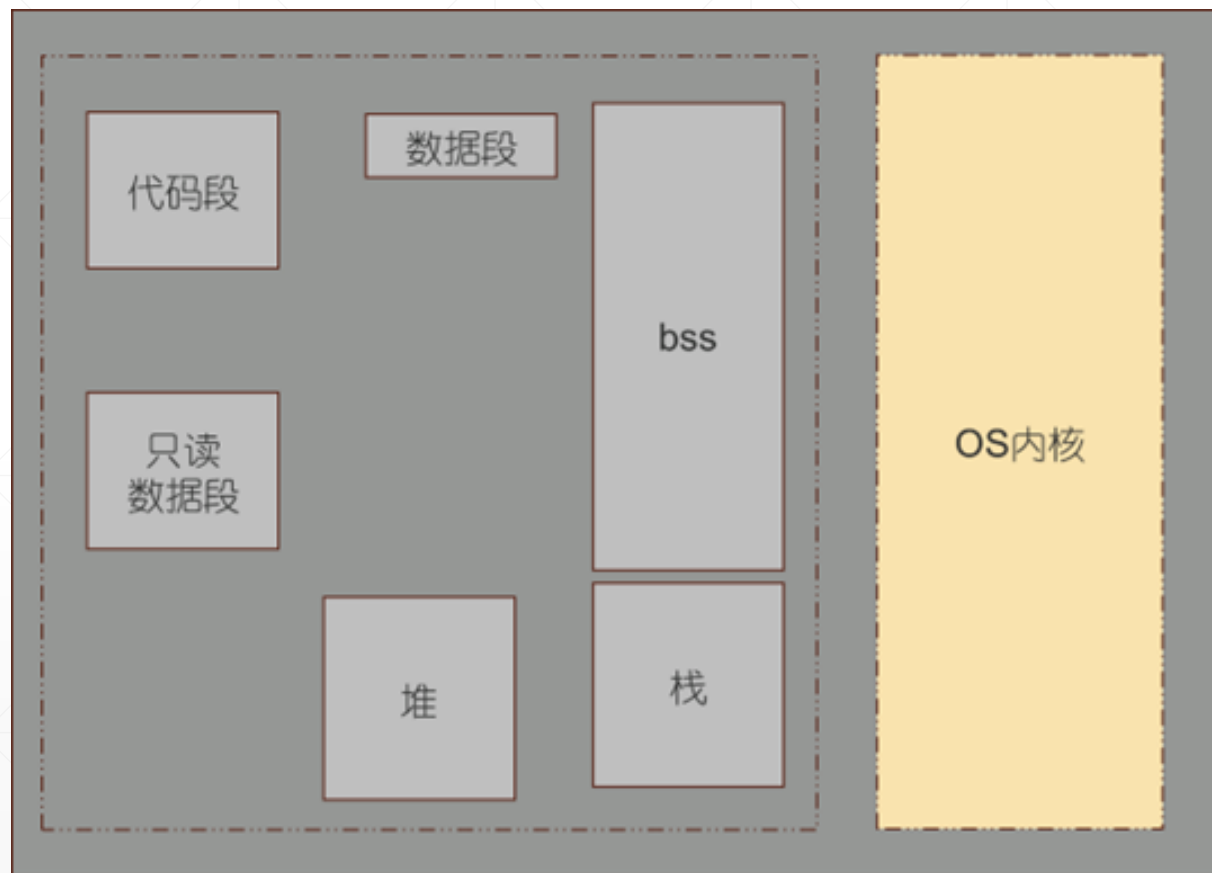


- 包括运行所需全部代码、数据
- 由属性不同的逻辑段组成

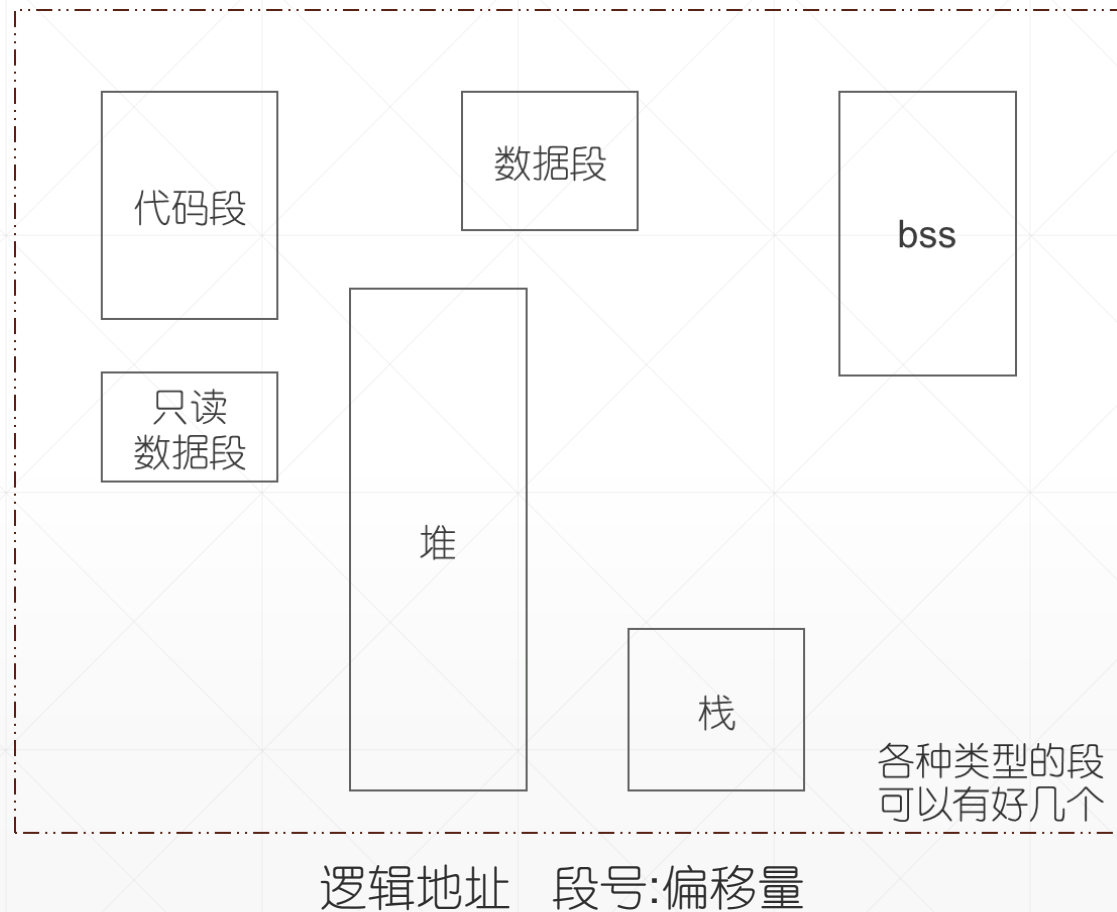
	逻辑段	保护位 prot			可执行文件中 有初值?	静态 vs 动态	扩展方向
		U/S	X	R/W			
用户空间	代码段	1	1	0	有	静态	---
	只读数据段	1	0	0	有	静态	---
	数据段	1	0	1	有	静态	---
	BSS 段	1	0	1	无	静态	---
	堆	1	0	1	无	动态	向高地址端
	栈	1	0	1	无	动态	向低地址端
核心空间	OS 内核	0	1	1	---	---	---

进程虚空间

- 多道系统为每个进程提供独用的虚空间，专门用来存放该进程的逻辑段。指令和数据的逻辑地址是其在虚空间中的地址。
- 虚空间由2部分组成
 - 用户区私有，存放应用程序的逻辑段，禁止其它进程访问
 - 核心区与其它进程共享，存放OS内核

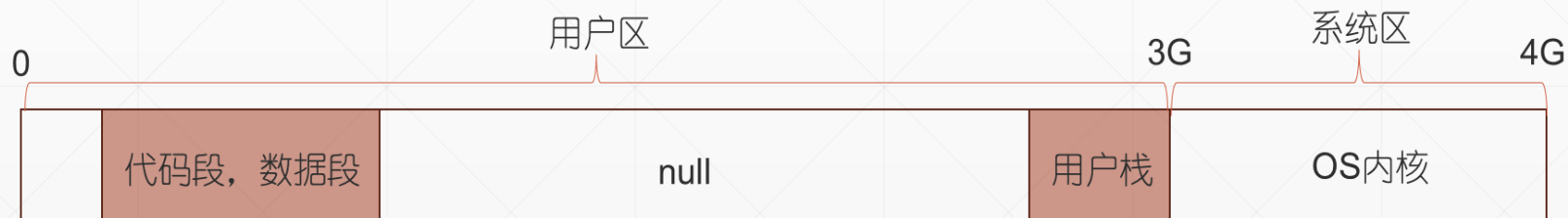


虚空间，一维 vs 二维 (用户空间为例)



例1：Linux进程的一维虚空间和逻辑地址

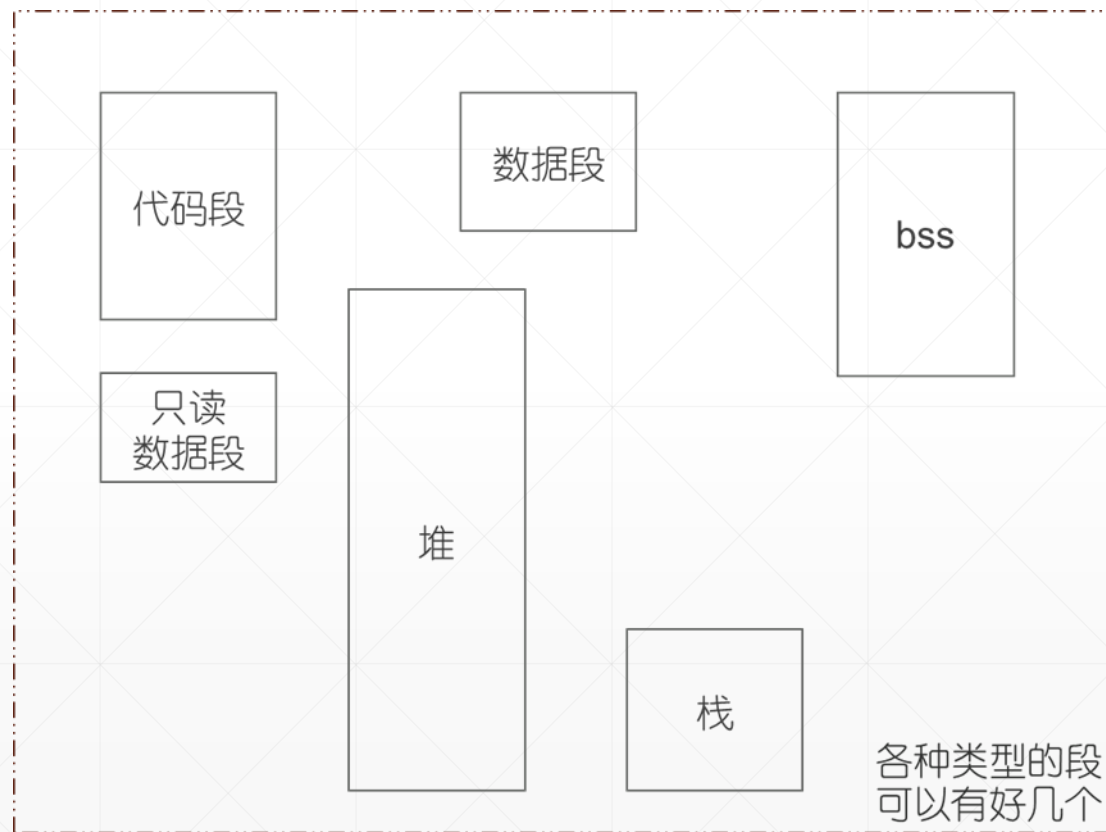
- 32位机，虚空间 2^{32} 字节，4G
- 逻辑地址 32位
- 代码段，第一条指令地址 0x08048000
- 数据段，第一个全局变量，地址 0x0804B000（如果代码段长度0x3000）
- 用户栈底，0xC0000000



例2：二维虚空间和逻辑地址

- 逻辑地址 段号：偏移量
- 32位机，偏移量32bit
- 代码段，第一条指令地址 0:0
- 数据段，第一条指令地址 1:0

现代计算机系统使用一维虚空间（线性地址空间），Linux/Windows皆如此。



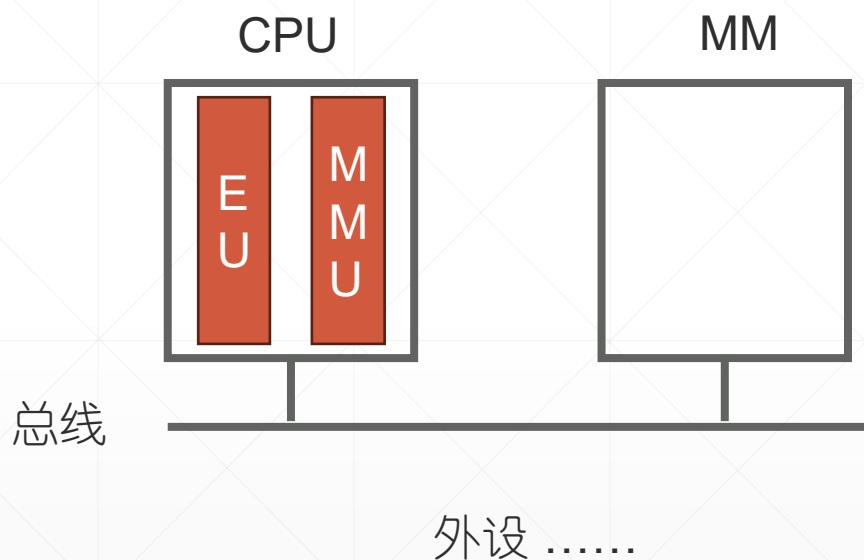
逻辑地址的分配

- 指令、全局变量、常量的逻辑地址，是编译过程分配的
- 用户栈底的逻辑地址，OS确定的（内核配置的常量）
- 局部变量 和 动态内存的逻辑地址，运行时分配
 - 局部变量，子函数调用、建立栈帧时分配
 - 动态内存，malloc函数分配
- 现代计算机系统，编译器生成可执行程序时，在虚空间中为所有指令、数据分配逻辑地址。这种不使用物理地址的应用程序可重定位，适合多道系统运行：
 - 内核加载应用程序时，可以为逻辑段分配任意内存块

可重定位的程序

- 现代计算机系统，编译器生成可执行程序时，在虚空间中为所有指令、数据分配逻辑地址（编址）生成可重定位的应用程序。程序执行时，
 - IP，逻辑地址
 - 指针的值，逻辑地址
 - 全局变量，地址常数，逻辑地址
 - 局部变量， $[EBP-X]$ ；入口参数， $[EBP+Y]$ 逻辑地址
- 多道系统，可以为可重定位程序的逻辑段分配任意内存块。
- CPU执行单元 取指、访问内存操作数 时，给逻辑地址。内存管理单元（MMU）

多道系统执行 可重定位程序 的过程



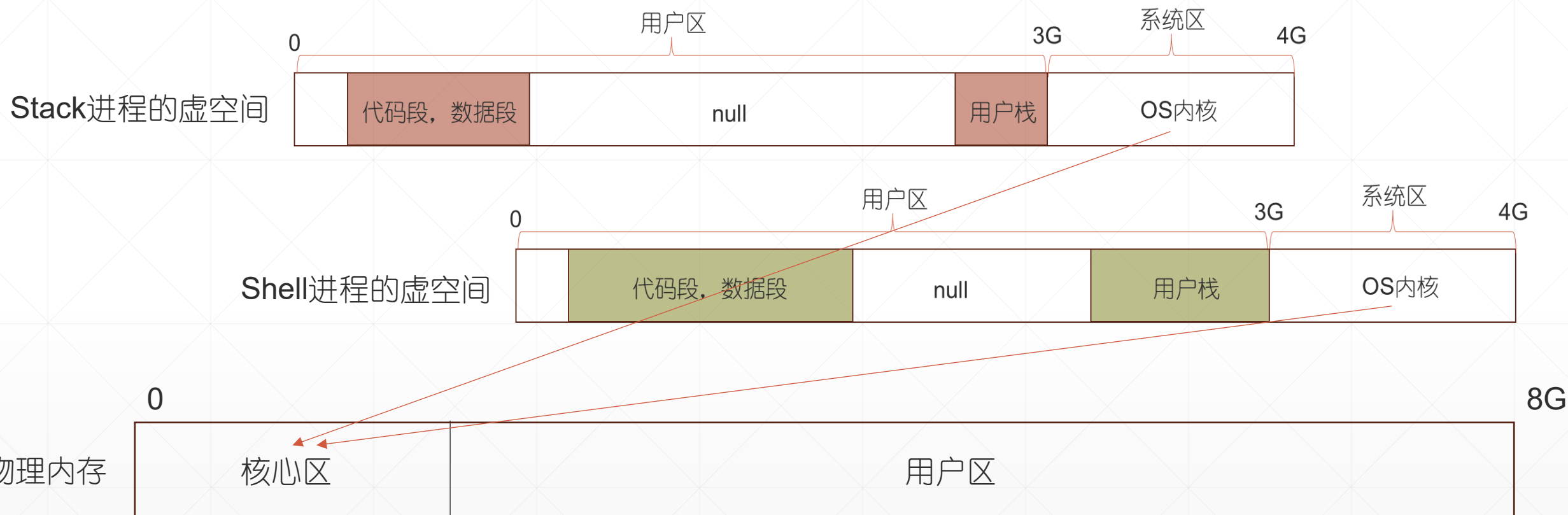
- 加载：为程序的逻辑段分配任意内存块 & 初始化
- 执行：CPU执行单元逐条执行指令。需要取指、访问内存操作数时
 - 执行单元（EU）给逻辑地址 VA,
 - 内存管理单元（MMU）将 VA 翻译成物理地址 PA,
 - 用PA访存。



二、多道系统，内存分配



物理内存核心区，线性映射虚空间中的OS内核



所有进程共享一份OS内核

物理内存用户区，4种不同的内存分配方式

一维虚空间

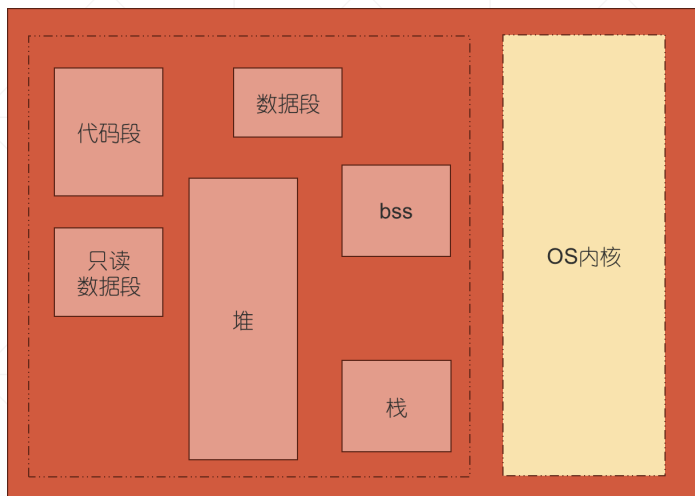
- 连续内存分配方式
 - 进程图像分配连续物理内存
 - 改良版，允许执行同一个应用程序的多个进程共享一份代码，Unix V6 (Unix V6++)
- 页式
 - 进程图像分页存放
 - 允许逻辑段内相邻页，物理内存不相邻

二维虚空间

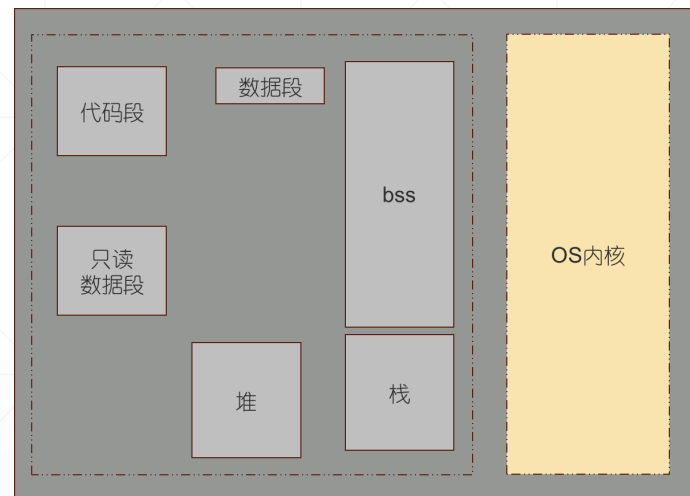
- 段式
 - 为每个逻辑段分配连续物理内存空间
 - 不要求所有逻辑段相邻
- 段页式
 - 段，分页存放于物理内存空间
 - 允许逻辑段内相邻页，物理内存不相邻

多道系统，内存分配

Shell进程图像



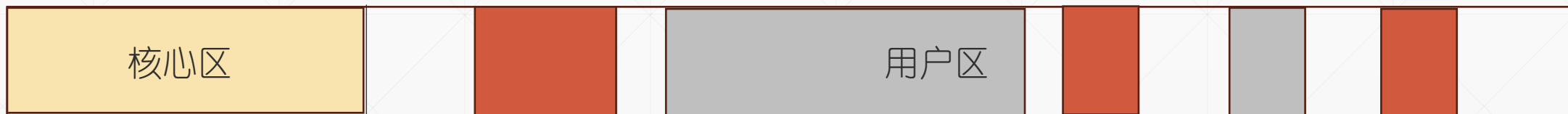
Stack进程图像



0

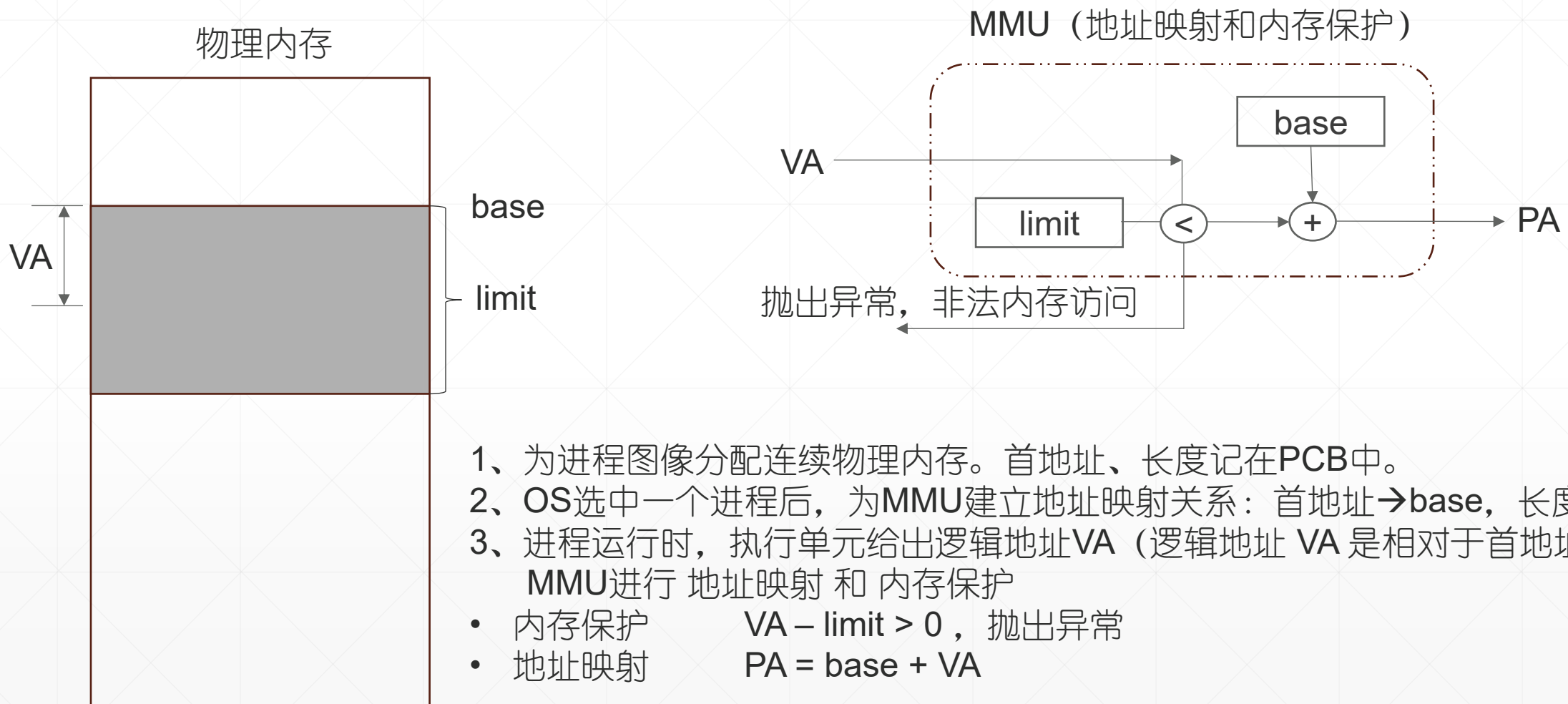
8G

物理内存



多道系统并发Shell进程和Stack进程。用户区为这两个进程的逻辑段分配物理内存，核心区共享。

2.1 连续内存分配方式（使用线性映射的MMU）



3种实现方式

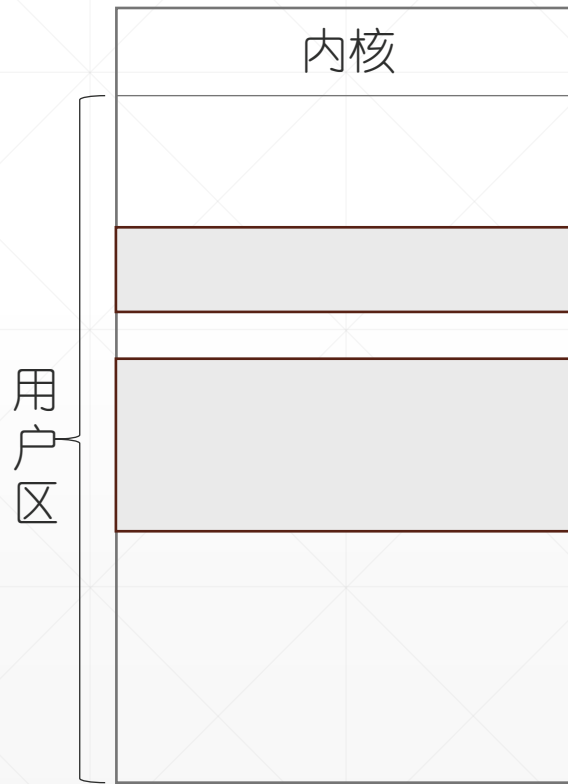
▪ 单一连续 (DOS)



▪ 固定分区



▪ 可变分区 (Unix V6+)



固定分区，分区的大小
长度、数量是固定的

可变分区，分区的大小
长度、数量不固定

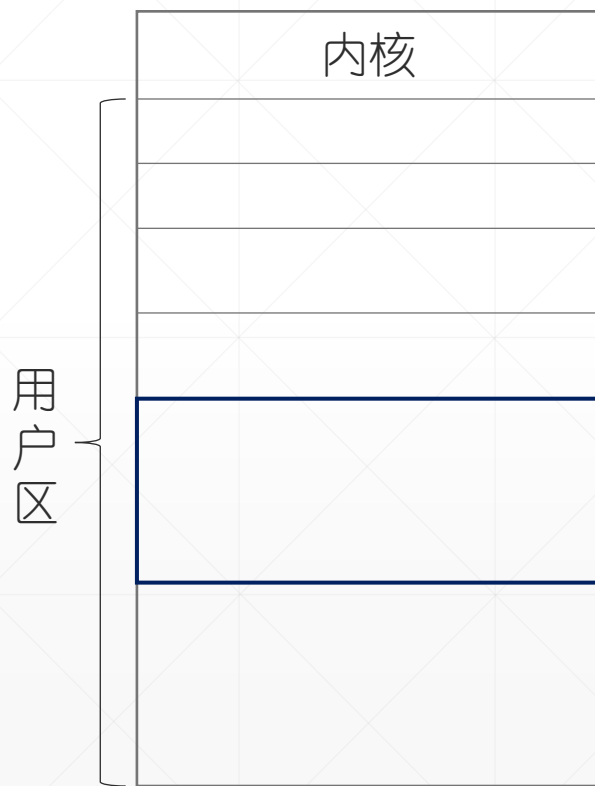
2.1.1 单一连续



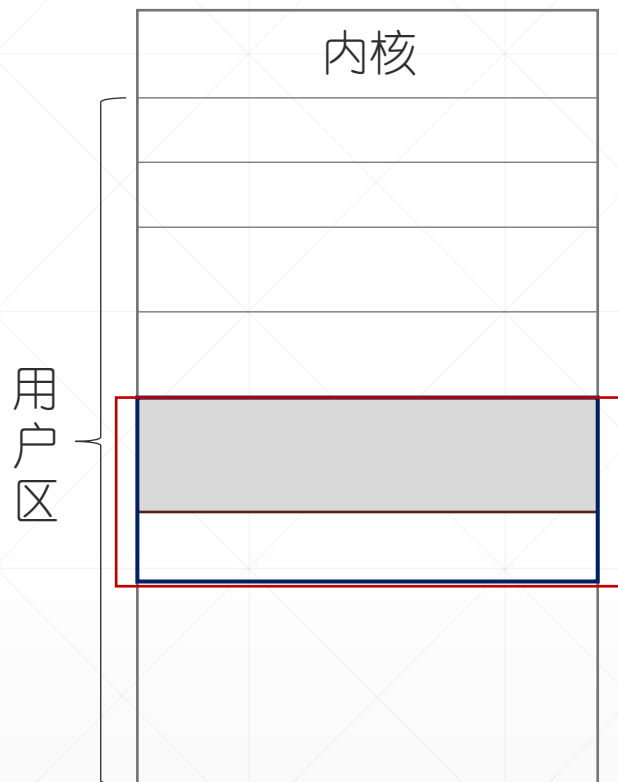
- 单道系统，用户区放一个应用程序。
- 优点：可以用物理地址编程。不使用MMU。

2.1.2 固定分区

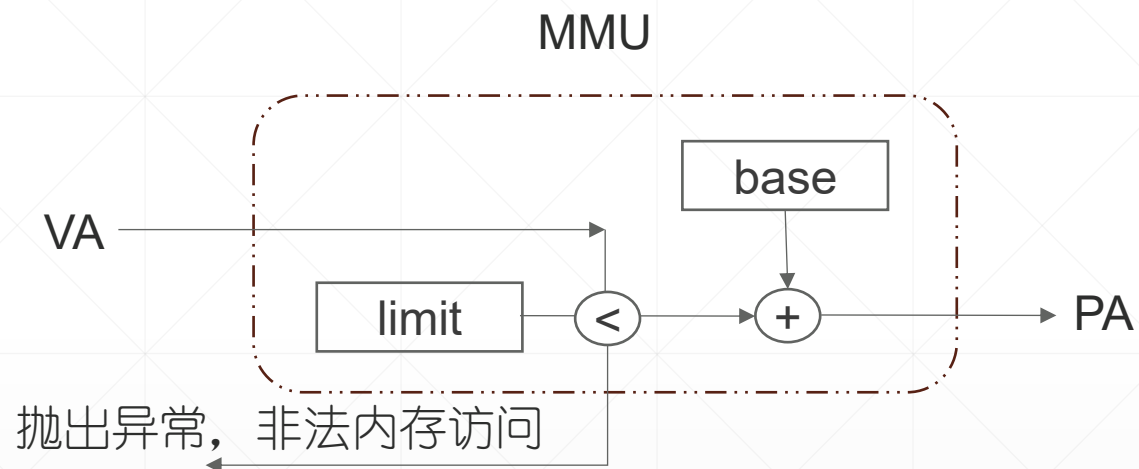
- 系统初始化时，划分分区。每个分区只能放一个进程。
- 系统维护**分区表**，登记所有分区的使用状态、首地址和长度。
 - 新建进程时，用掉一个分区，标记占用。
 - PCB: 进程起始地址 = 分区起始物理地址
进程长度 = 进程图像实际长度
分区表下标ID
 - 进程终止时，释放分区，标记空闲。
 - 分区表, ID表项, 空闲位置1



ID	起始物理地址	长度	空闲位
			
			



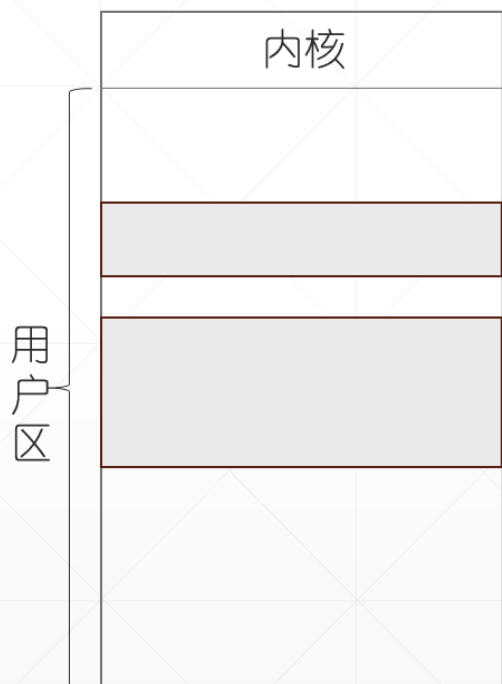
- 为现运行进程建立地址映射关系
PCB中起始地址 $\rightarrow$ base, 长度 $\rightarrow$ limit



- 缺点: 会产生块内碎片, 降低物理内存利用率。

2.1.3 可变分区

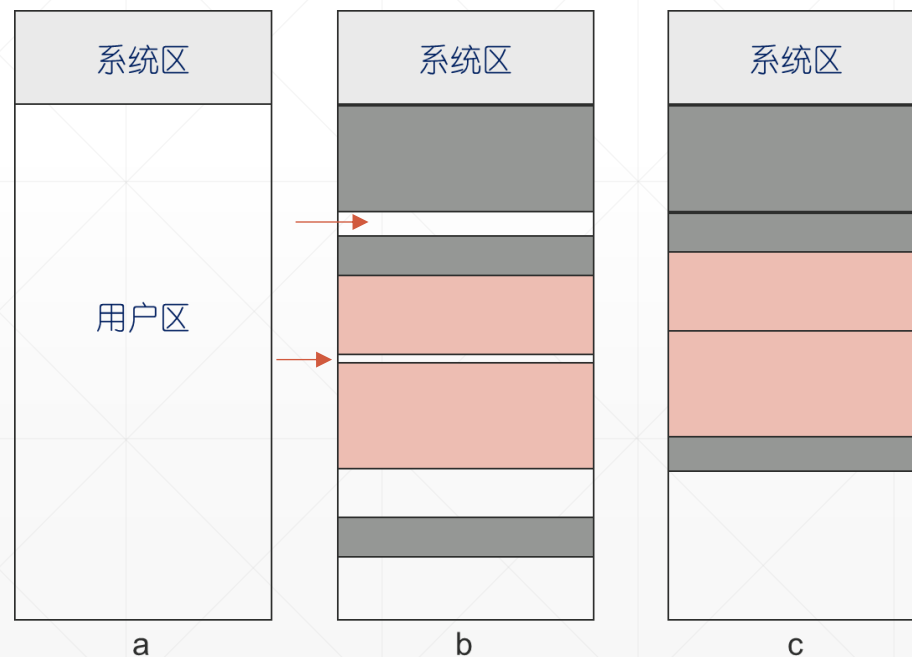
- 每个分区放一个进程。分区不固定。
- **用空闲分区表** (map数组) 登记所有空闲分区的首址和长度。



a) 可变分区管理方式，一开始内存使用状态良好。后来会产生好多不容易利用的小分区，叫做外部碎片。

b) 外部碎片装不进任何进程图像，会降低物理内存利用率。

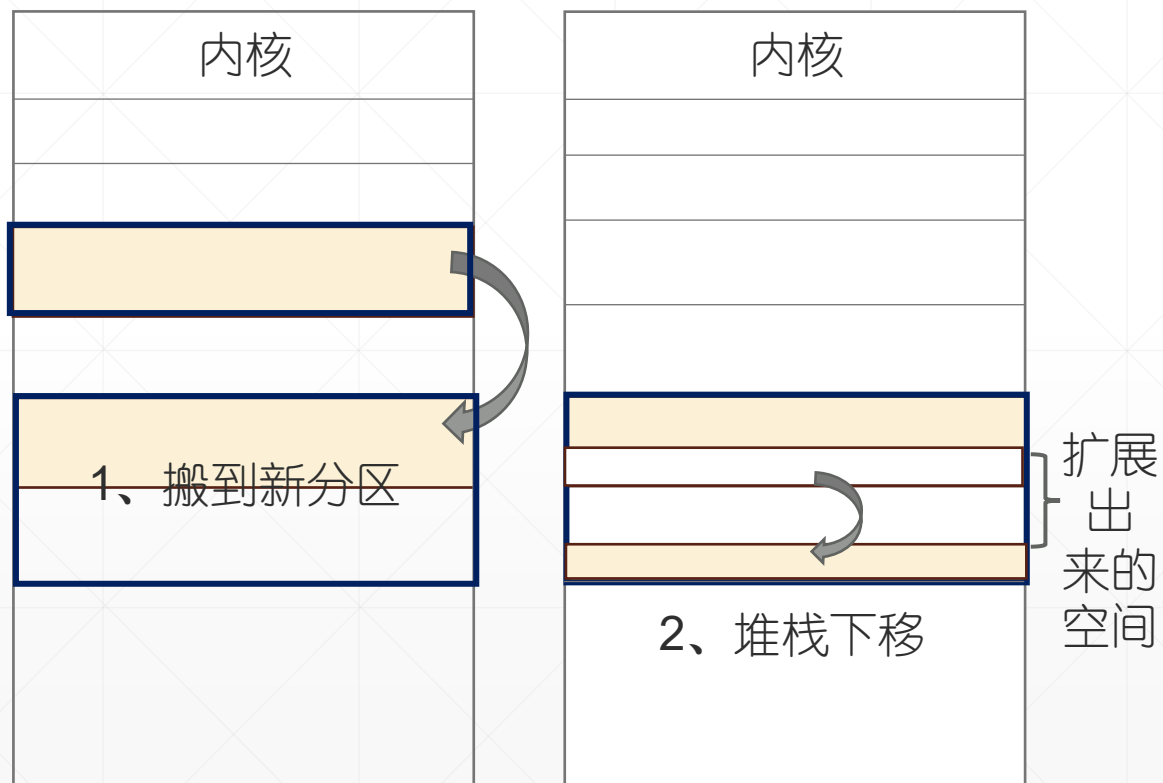
c) 解决外部碎片问题，可以采用拼接技术：平移进程图像，以合并空闲分区。开销巨大。



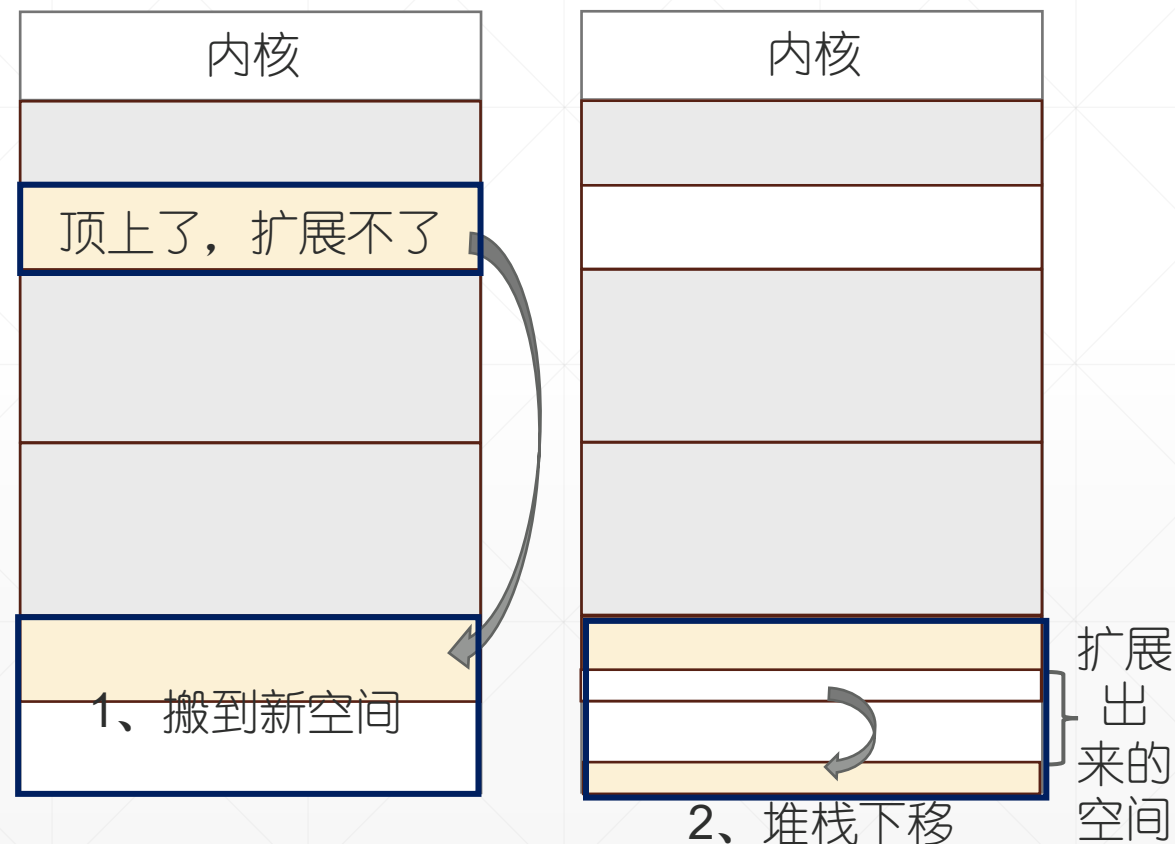
2.1.4 痛点1 进程图像很难扩展

原来的分区容纳不了扩展后的图像
进程图像要全部平移，开销巨大

■ 固定分区



■ 可变分区 (Unix V6++)





2.1.5 痛点2 碎片降低物理内存利用率

痛点1、2，优化方案： 进程图像离散存放

2.1.6 进一步的优化空间

- 连续存储管理方式，要求运行前进程图像全部在内存。具体体现在：
 - 进程图像整体移动，swap in、swap out 速度慢。
 - fork，为子进程复制父进程全部图像
 - exec，加载、初始化所有逻辑段后，应用程序才能开始执行
- 但是，运行的每个阶段，程序只会访问部分进程图像（比如固定的若干子程序，或是数组的某个片段）。即：最近访问过的代码和数据，未来还会访问；最近访问过的区域，未来会访问它附近的区域。这是程序运行的局部性原理。
- 考虑程序运行的局部性原理，运行前不必加载全部进程图像。

总结、对连续存储管理方式的评论

- 优点：管理起来方便。PCB中只需要登记进程图像在物理内存中的起始地址和长度
- 缺点：
 - 存在碎片问题
 - 进程图像扩展开销巨大
 - 没办法利用程序运行局部性原理降低内存管理开销
- 克服缺点的方法：
 - **离散存储管理方式**。解决碎片问题，提高物理内存资源利用率；还能够降低进程图像扩展开销。
 - **虚拟存储器技术**，让有限内存可以容纳更多进程，可以进一步提升系统整体性能。具体思路如下（1）盘交换区和文件系统，保存所有逻辑页。（2）内存用来存放进程近期需要访问的逻辑页。（3）请求调页。其余页面，访问时，缺页异常处理程序为其分配内存，从磁盘调入。

关于 Unix V6++ 的讨论

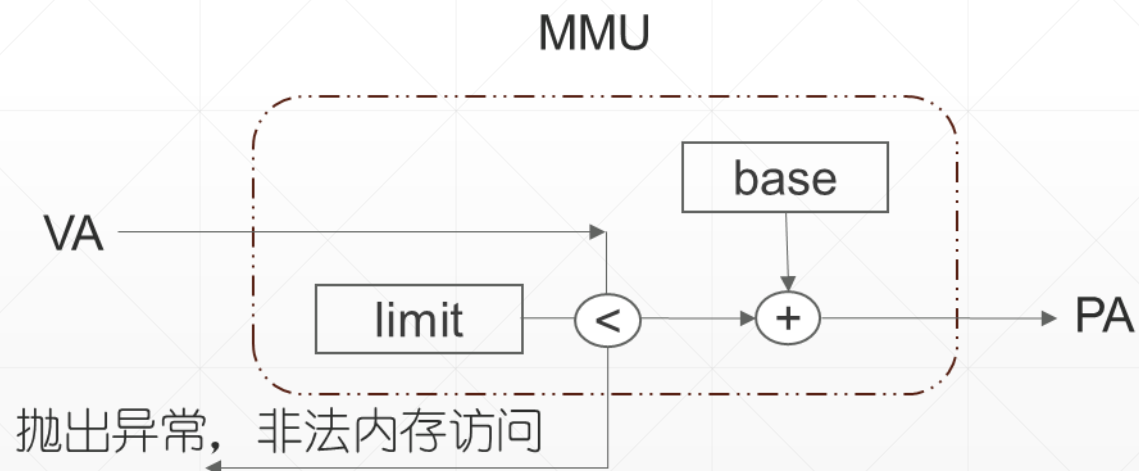
- Unix V6, Unix V6++ 是连续存储管理方式
 - 为正文段分配连续物理内存
 - 为可交换部分分配物理内存
 - 正文段与可交换部分不相邻, 以利执行同一个应用程序的多个进程共享正文段
- 继承连续存储管理方式的所有优缺点。
- 借助页表, 将虚空间中不相邻的逻辑段映射至物理空间相邻的位置。这是为了避免为不存在的虚空间分配物理内存。

典型例题 1

1、连续内存管理方式，访问指令或数据，需要几次访存？

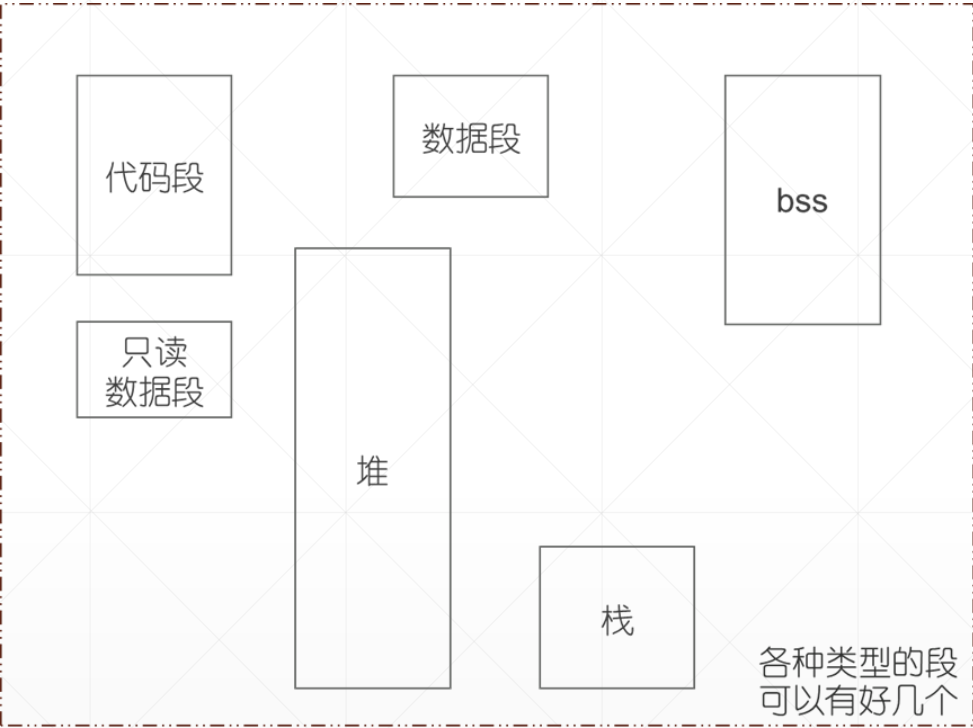
答：1次。计算物理地址无需访存。访问指令或数据，1次访存。

2、下图所示MMU单元不支持Unix V6++的地址映射操作，为什么？





3、段式内存管理方式



段表

base	limit	prot
0x500	0x0b0	U, RO, X
0x300	0x150	U, RW, ~X
.....	0x050	U, RW, ~X
.....	0x020	U, RO, ~X
.....	0x1000	U, RW, ~X
0x700	0x3000	U, RW, ~X

实空间
(物理内存)



为每个逻辑段独立分配物理内存空间。
每个进程一张段表。登记所有逻辑段的首地址，长度和保护位。

地址映射 和 内存保护

逻辑地址： 段号

段内偏移量 (offset)



段表寄存器

起始地址 (物理)	长度 (6)
-----------	--------

段表
(每个进程一个, 在PCB里)

base	limit	prot
0x300	0x0b0	U, RO, X
0x400	0x150	U, RW, ~X
0x600	0x050	U, RW, ~X
0x800	0x020	U, RO, ~X
0xa00	0x1000	U, RW, ~X
0x2000	0x3000	U, RW, ~X

段号
0
1
2
3
4
5

- 1、CPU内部有一个段表寄存器, 引用现运行进程的段表
- 2、CPU EU给出的逻辑地址是2维的, 包括 段号 和 段内偏移量
- 3、MMU 地址映射 和 内存保护 逻辑如下:

- 段号 $\geq$ 段表寄存器中的长度, 异常
- 用段号做下标查段表, 得到一个段表表项
 - $\text{offset} > \text{limit}$, 异常
 - 写操作, prot是RO, 异常
 - 取指操作, prot是~X, 异常
- 物理地址 = base + offset

内存保护

地址映射

典型例题 2

1、段式存储管理，访问指令或数据，需要几次访存？

答：2次。一次访问段表，生成物理地址。第二次，访存，访问指令或数据。

2、段式存储管理系统，地址长度32位，其中段号占8位。问最大段长是？（2009）

答： 2^{24} 字节。

逻辑地址： 段号

段内偏移量 (offset)



3、【2016 统考真题】某进程的段表内容如下所示。

段号	段长	内存起始地址	权限	状态
0	100	6000	只读	在内存
1	200	—	读写	不在内存
2	300	4000	读写	在内存

访问段号为 2、段内地址为 400 的逻辑地址时，进行地址转换的结果是（ ）。

- A. 段缺失异常
- B. 得到内存地址 4400
- C. 越权异常
- D. 越界异常

段是自然的共享、保护单元

- 2个进程共享一个代码段 和 一个只读数据段

PA段表
(每个进程一个, 在PCB里)

段号	base	limit	prot
0	0x500	0x0bf	RO, X
1	0x300	0x15f	RW, ~X
2	0x480	0x1f	RO, ~X
3	0x700	0x0ff	RW, ~X

PB段表
(每个进程一个, 在PCB里)

段号	base	limit	prot
0	0x500	0x0bf	RO, X
1	0x800	0x15f	RW, ~X
2	0x480	0x1f	RO, ~X
3	0xa00	0x0ff	RW, ~X

特别地,
共享库 (动态链接库) 的代码段, 整个系统配一份, 所有进程只读共享

- 2个进程共享同一个数据段

PA进程的段表

段号	base	limit	prot
0	0x500	0x0bf	RO, X
1	0x300	0x15f	RW, ~X
2	0x480	0x200	RW, ~X
3	0x700	0x0ff	RW, ~X

PB进程的段表

段号	base	limit	prot
0	0x500	0x0bf	RO, X
1	0x300	0x15f	RW, ~X
2	0x700	0x0ff	RW, ~X
3	0x480	0x200	RW, ~X

共享数据段, 物理地址 [0x480, 0x680)

访问该数据段, 使用的逻辑地址:

PA [2: 0, 2: 0x200)

PB [3: 0, 3: 0x200)

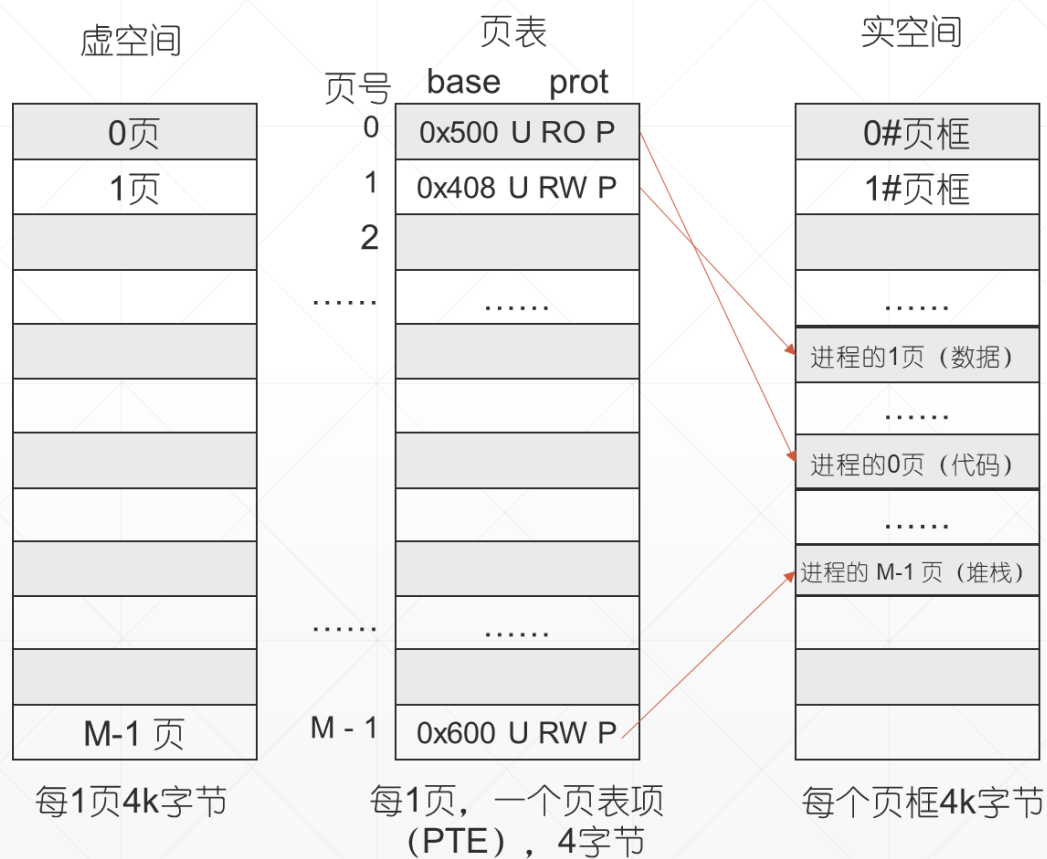
一个进程做的更新, 另一个进程马上可见。这是最快的进程通信方式。
两个进程, 同时更新一个位置, 值可能会错。要互斥。



段式内存管理 的 内存分配

- 内存分配，以自然段为单位。自然段长度可变，内存分配麻烦。存在外部碎片问题。物理内存利用率依然很低。
- 比连续存储管理方式好的地方：堆扩展，用户栈扩展时，其余逻辑段不必移动。

六、页式存储管理方式



虚空间等分成若干逻辑页面。逻辑页面 2^n 字节。
实空间（物理内存），按相同尺寸等分成若干物理页框。
为逻辑页面分配任意空闲物理页框。即，不必为相邻逻辑页分配相邻的物理页框。

每个进程一张页表（**Page Table**），是一个数组。
每个逻辑页面对应一个页表项（pte, Page Table Entry）：
base 登记分配给它的物理页框号。

prot, 是保护位：

- U/S, 1(U)是应用程序的逻辑页, CPU用户态运行时有权访问
0(S)是内核的逻辑页, CPU用户态运行时无权访问
- RW/RO, 1(RW) 可读、可写
0(RO) 只读
- X, 1、代码页, 可执行
0、数据页, 不可执行

逻辑地址 和 页表

逻辑地址，一维。

例：16位虚地址

逻辑页号 P

页内偏移量 offset



页表寄存器

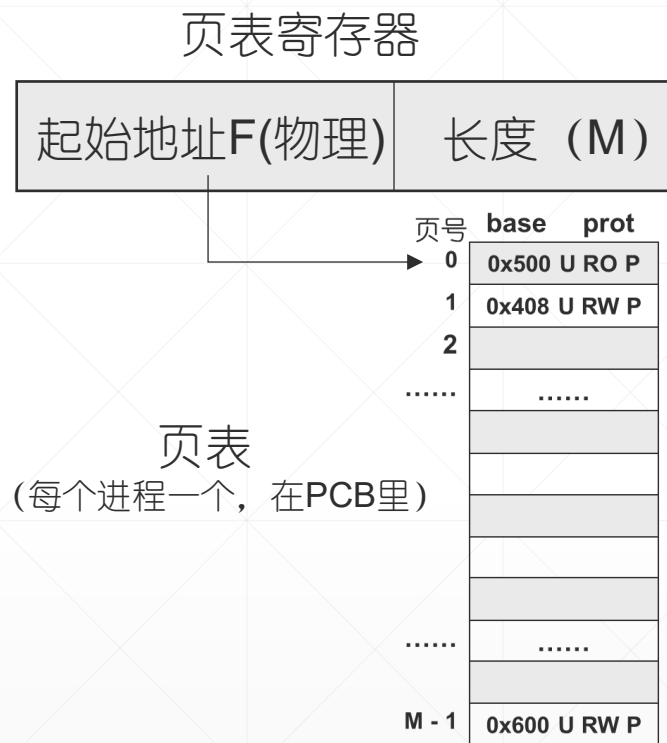
起始地址F(物理)	长度 (M)
-----------	--------

页号	base	prot
→ 0	0x500	U RO P
1	0x408	U RW P
2		
.....		
.....		
M - 1	0x600	U RW P

页表
(每个进程一个, 在PCB里)

- 1、CPU内部有一个页表寄存器，引用现运行进程的页表
- 2、CPU给出的逻辑地址是1维的。
 - 低位n bit是页内偏移量 offset
 - 高位的其余bit 是逻辑页号 P
- 3、MMU用逻辑地址查页表，实施地址映射 和 内存保护操作

地址映射和内存保护，MMU的硬件逻辑



- 将逻辑地址分割成 页号P 和 页内偏移量offset
- $P \geq M$, 异常 (缺页中断)
- 用P做下标查页表, 得到一个页表项pte
 - 页表项的物理地址 = $F + P * \text{页表项长度}$
 - pte为null, 异常 (缺页中断)
 - 写操作, prot是RO, 异常 (缺页中断)
 - CPU用户态运行, prot是S, 异常 (缺页中断)
- 物理地址 = $\text{base} * L + \text{offset}$ (L是页面的长度)
 - 实现上, 用base拼接offset

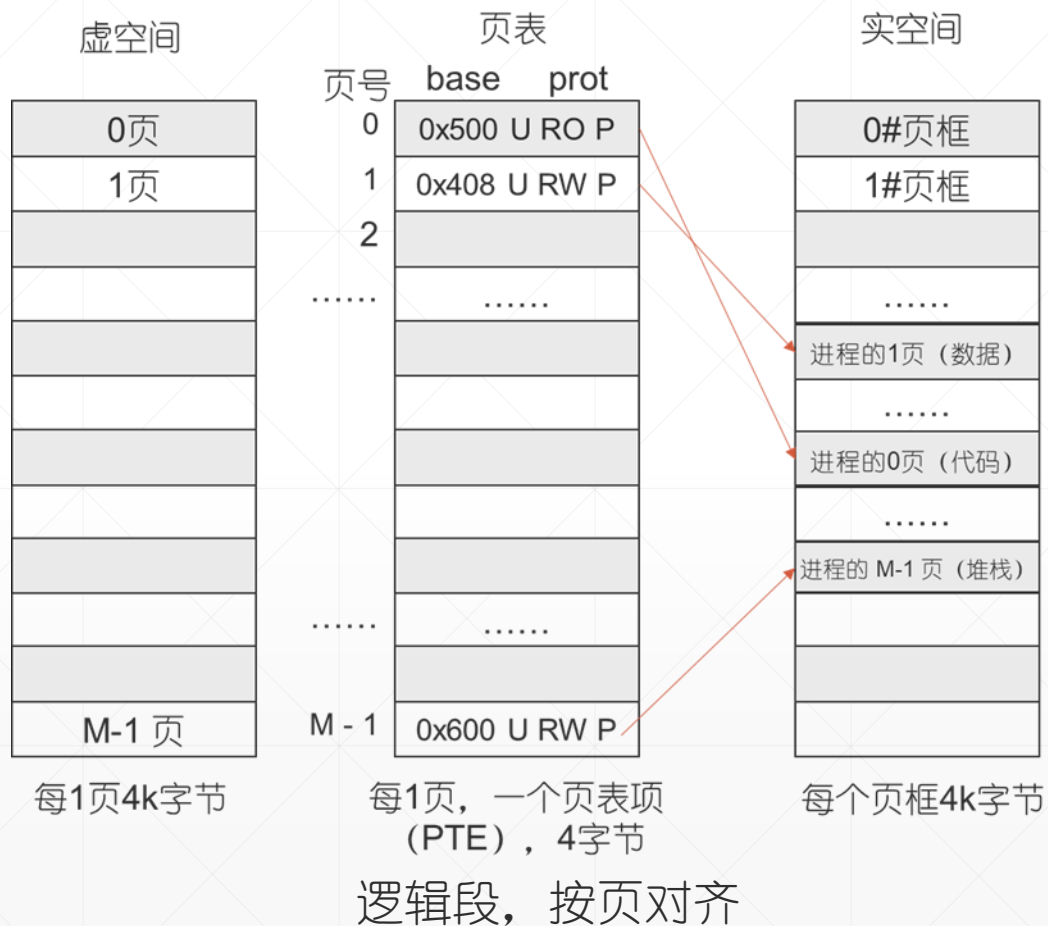


典型例题 3 页式内存管理，地址映射

1、在一个页式存储管理系统中，页表内容如下所示。若页的大小为4K，则地址转换机构将逻辑地址 0x1025 转换成的物理地址为 (0x3025) 。

页号	内存块号
0	2
1	3
2	8

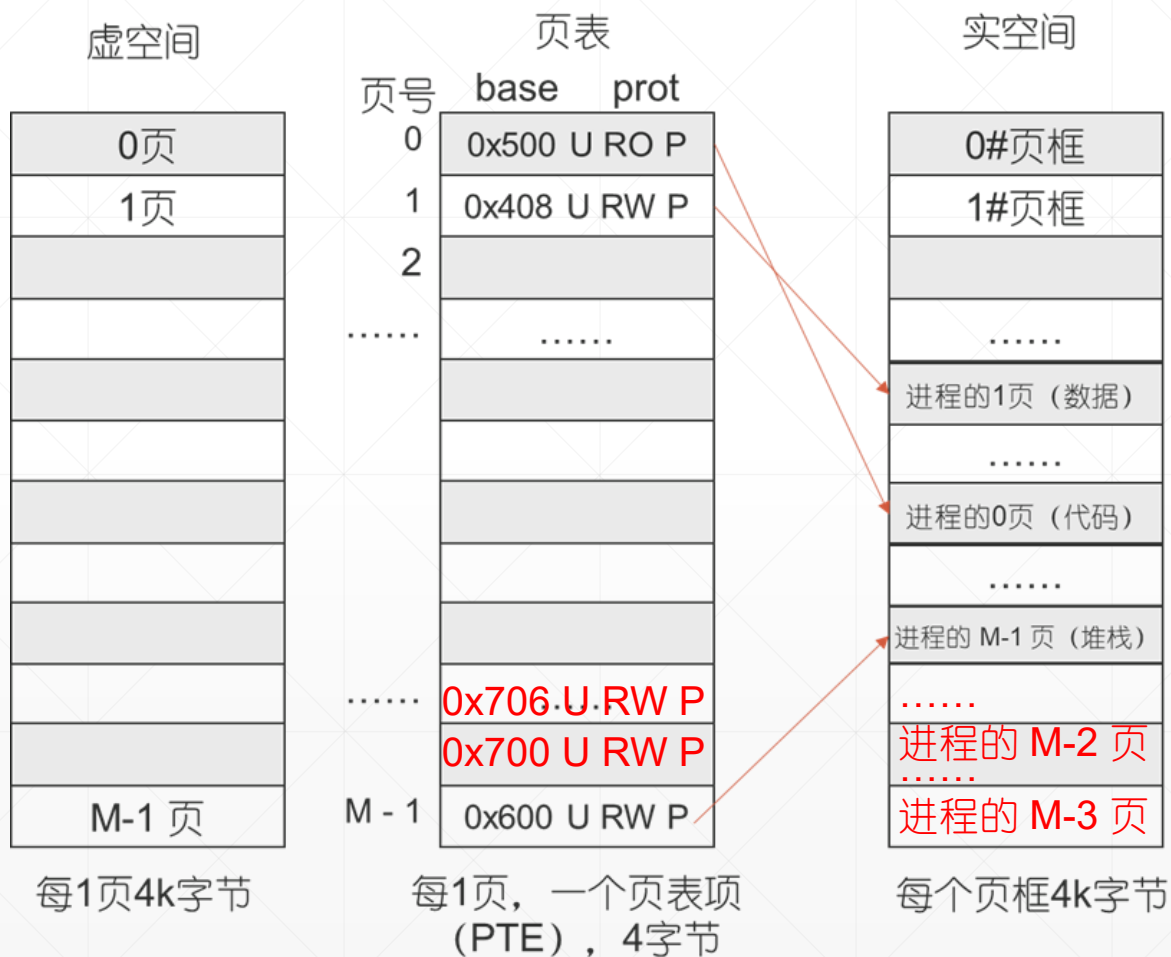
页式存储管理的优点 1



- 页式, 最显著的优点, **提高物理内存的利用率。**
- 原因: 每一个物理页框可以用来存放任何进程的任意逻辑页面。不存在外部碎片问题。
- 需要注意: 页内碎片是有的, 每个逻辑段的末页, 平均浪费半个页框的存储空间 (2K字节)。

页式存储管理的优点 2

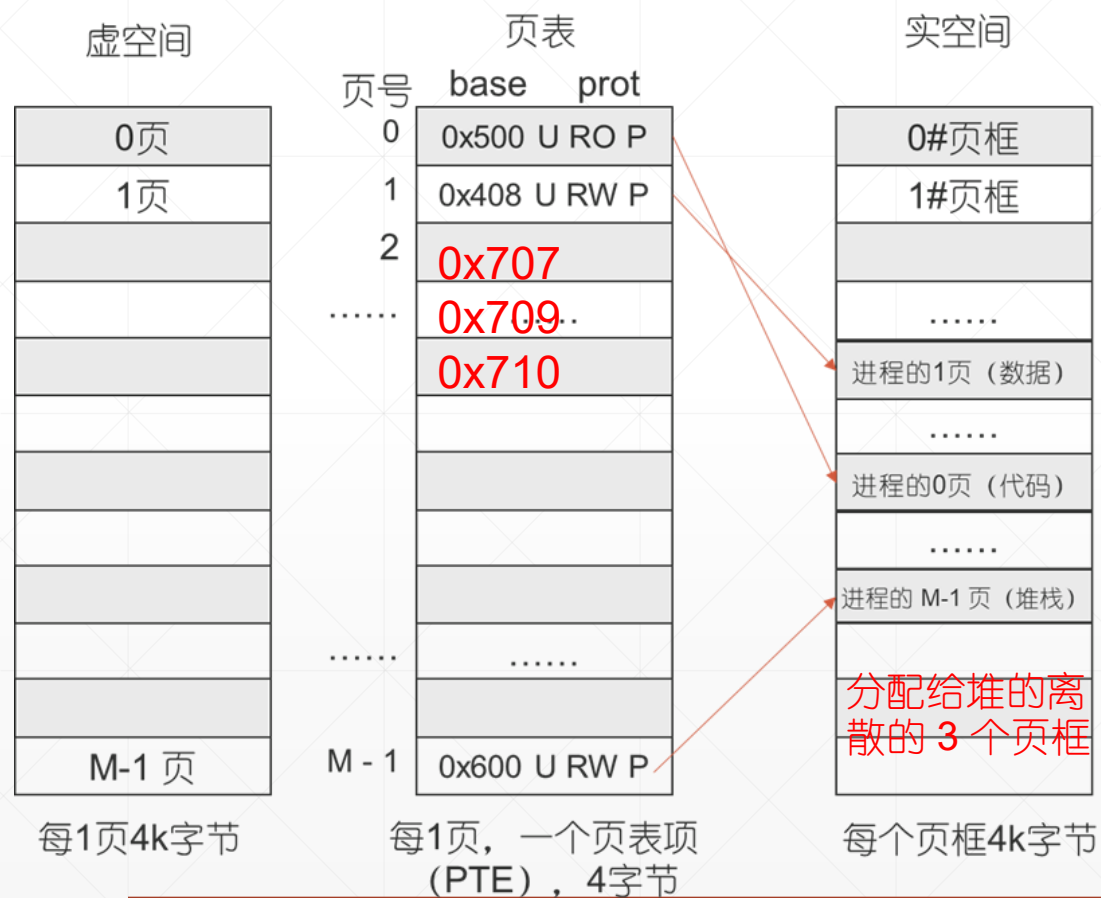
扩展进程图像极其容易，已有进程图像不必移动



例：stack扩展 2页

- 找2个空闲的物理页框，比如 0x700#，0x706#
- PCB找到内存描述符，找到stack长度（A个页面）。
- PCB找到页表，填写下标 M-A-1的 PTE
 - base = 0x700
 - prot = U RW P
- 填写下标 M-A-2的 PTE
 - base = 0x706
 - prot = U RW P
- 内存描述符，堆栈段长度+2（=A+2）

扩展进程图像极其容易 (续)



heap扩展 3页

- 找3个空闲的物理页框，比如 0x707#，0x709#，0x710
- PCB找到内存描述符，找到 heap起始虚地址（页号是X，本例是 2）长度（B个页面，本例是0）。
- 填页表，下标 X+B 的 PTE
 - base = 0x707
 - prot = U RW P
- 填页表，下标 X+B+1，X+B+2的 PTE
 -
- 内存描述符，heap长度+3 (=B+3)

对比，连续存储管理方式，堆栈扩展 1 页 (Unix V6++为例)

烦死啦

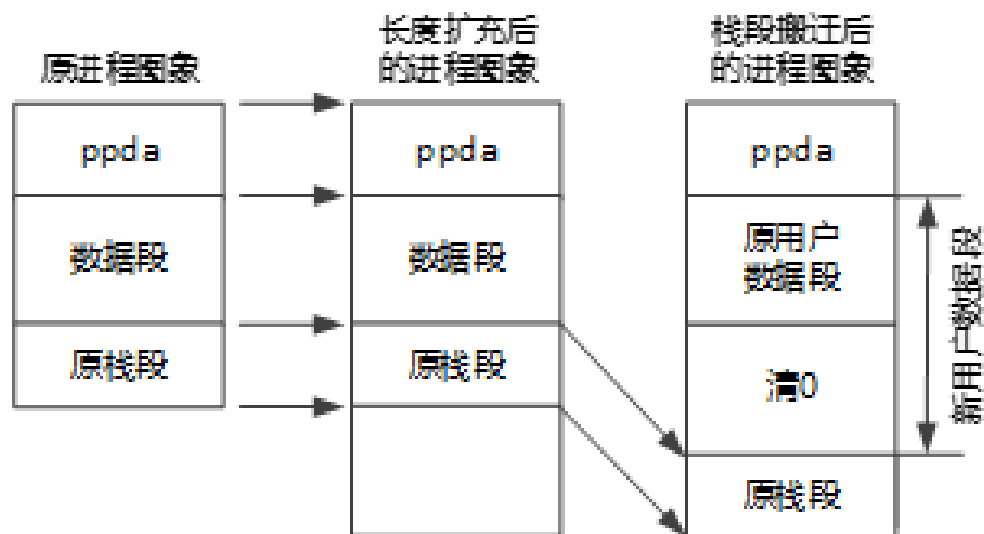


图 5.24：用户数据段的扩充

- 1、按扩充后的尺寸申请一块新空间，更新 `p_addr` 和 `p_size`
- 2、修改内存描述符，更新堆栈段尺寸
- 3、修改相对虚实地址映射表，反映新的相对地址映射关系
- 4、可交换部分整体复制到新空间
- 5、在新空间内部，将原栈段平移至底部
- 6、`MapToUserPageTable`，把新的地址映射关系写入系统页表

页式存储管理的优点 3，以页为单位共享和保护物理内存

PA

base	prot
0x401	RW.....

PB

base	prot
0x401	RW.....

页面长度4096字节。

提问：访问物理页框0x401的0#字节。

PA使用逻辑地址 () ?

PB使用逻辑地址 () ?

以页为单位共享和保护物理内存，相较段式存储管理，维护起来麻烦不少。

- 大段数据要共享，要设置好多PTE，把 base 和 prot 写成相同的值。

128
 {

base	prot
401	RW.....
500	RW.....
.....	
.....	
the same	

base	prot
401	RW.....
500	RW.....
.....	
.....	
the same	

PA

base	limit	prot
.....		
0x300	0x800000	RW, ~X
.....		
.....		

PB

base	limit	prot
.....		
.....		
0x300	0x800000	RW, ~X
.....		



典型例题 4 页表的内存消耗

2、已知某机器，虚空间 4G 字节。页面的尺寸是 4K 字节。pte, 4 字节。

虚空间有 $4G/4K = 1M$ 个逻辑页。

页表，有 1M 个页表项。每个页表项 4 字节。页表大小是 4M 字节。

每个进程页表开销 4M 字节，且必需占据连续物理内存。**页表的内存开销不可忍受。**

页式。指令执行的耗时。

- **指令执行的耗时：取值 + 取操作数 + 执行 + 存运行结果**

- **取值，1次访存**

- **取操作数**

 - 立即数 或 寄存器，不访问内存，耗时极短，可以忽略不计。

 - **内存操作数，1次访存**

- CPU内部指令执行的耗时，一般远远小于访存耗时。

- **存运行结果**（与 取操作数相同）

性能瓶颈 在 访存。

- **每次访存耗时：地址映射 + 存/取操作数**

- **地址映射**

 - 取PTE，一次访存

 - 虚地址切割，1个CPU cycle

 - 拼接物理地址，1个CPU cycle

} 可忽略

- **存/取操作数，一次访存**

结论：页式地址映射，访存耗时加倍。

CPU运算速度减半。

总结：分页机制存在2个主要问题

- 访存速度降低。因为地址映射要取PTE。
- 每个进程一个页表。页表需要消耗大量连续物理内存。

解决问题的方法

- 降低页表内存消耗，用二级、多级页表。
 - 若大片虚空间是空，无需为其设置页表。PTE不存啦。
 - 结合虚拟存储，请求调入页表，为目前需要使用的页表分配物理页框，其余页表留存盘交换区、甚至延迟创建。
- 访存速度降低。CPU加TLB单元。

降低页表的内存开销，第一种方法：多级页表 (Intel)

- 二级页表 (i386)。

- 页目录 和 页表，占据一个物理页框。
- 每个物理页框 4096字节。页表项PTE (页目录项PDE) 4字节 (base 20位, prot 12位)。

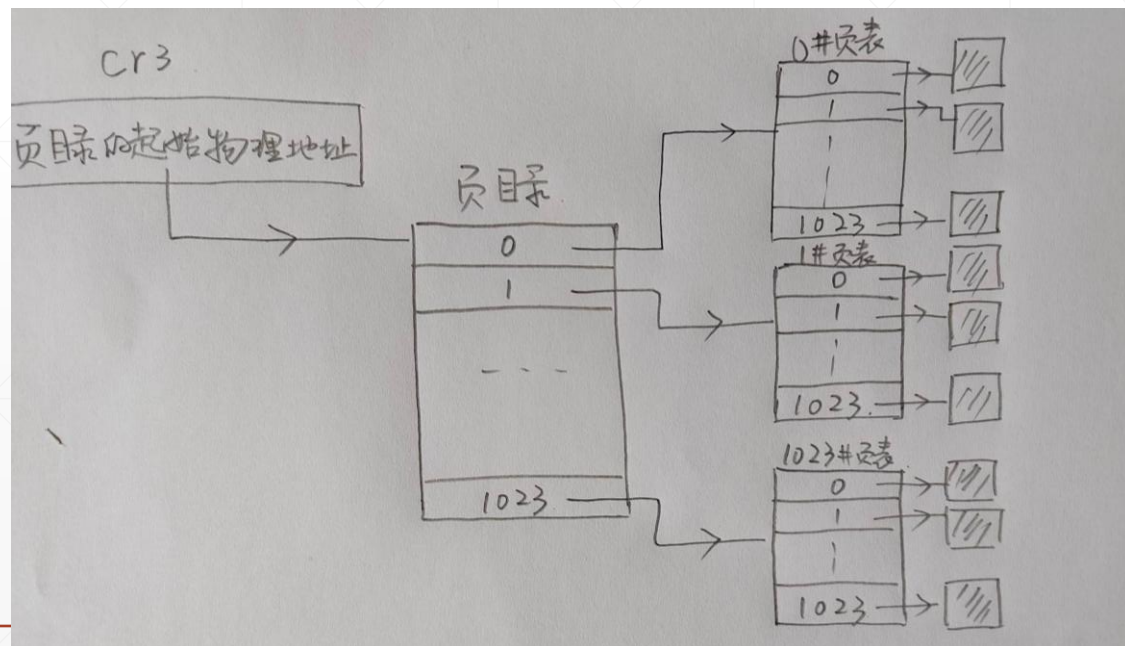
∴ 页目录1024项，每一项管理一张页表

每张页表1024项，每一项管理一个逻辑页 (4096字节)

∴ 每个页目录项管理连续1024张逻辑页，共计4M字节虚空间

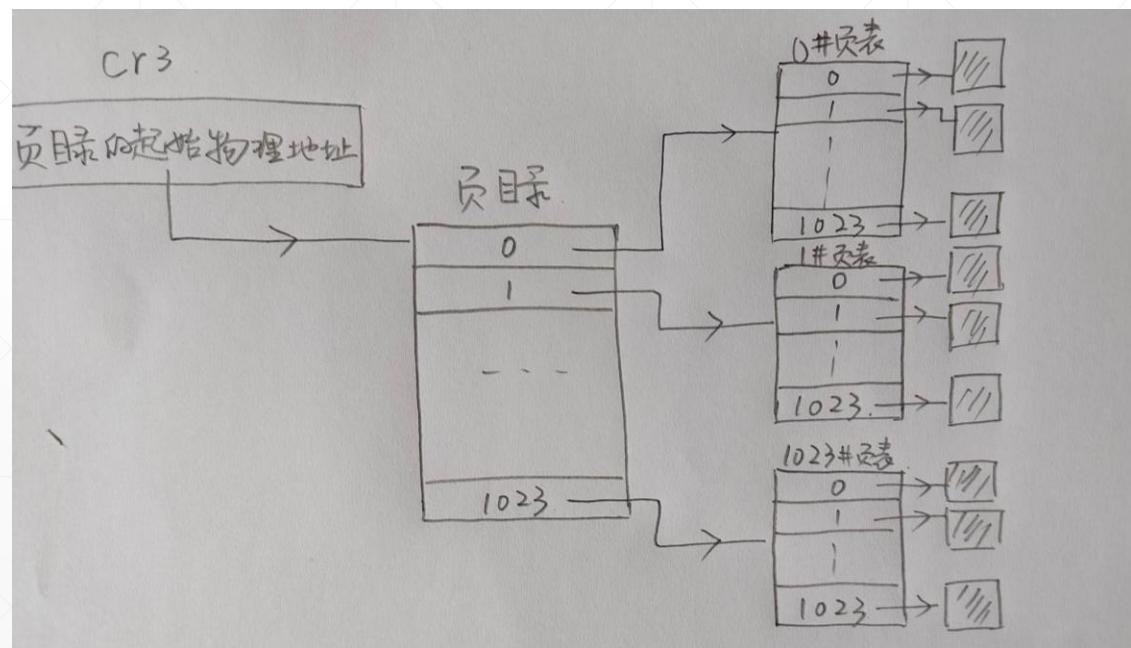
如果，这段虚空间为空。没必要分配页表，PDE置空。

页表开销降低4K字节。

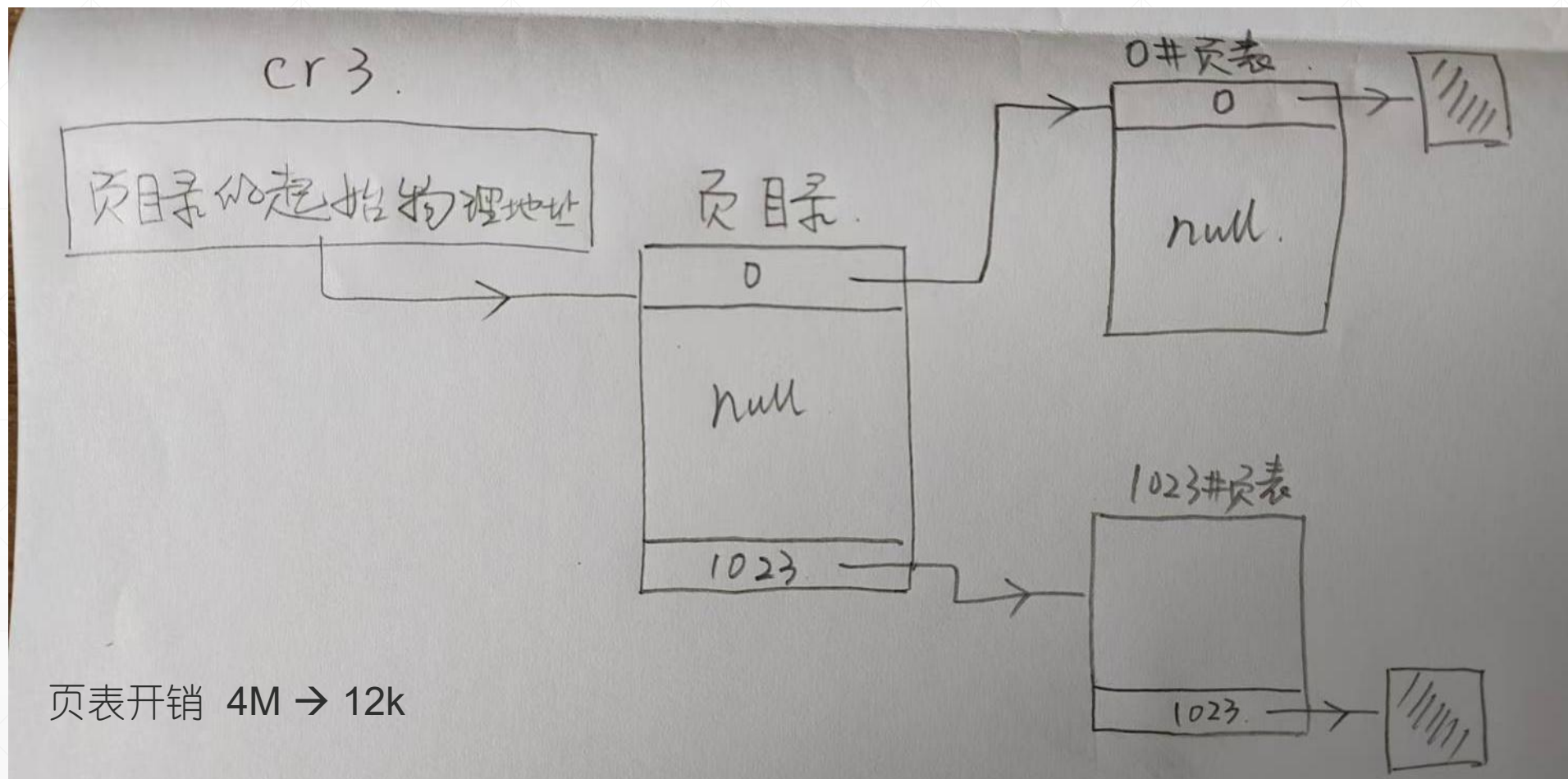


降低页表的内存开销，第一种方法：多级页表 (Intel)

- 二级页表 (i386) 。
 - 页长 4096字节，每个物理页框 4096字节。PTE(PDE) 4字节，base 20位，prot 12位。
 - 页目录 和 所有页表， 占据一个物理页框。
- 32位虚地址
 - 高10位查页目录
 - 低10位查页表
 - 最后12位是页内偏移量
- 每次取值/访问内存操作数， 3次内存访问
 - 访问页目录， 取PDE
 - 访问页表， 取PTE
 - 访问逻辑页， 存取物理内存



最小进程（一页代码，一页堆栈）的二级页表



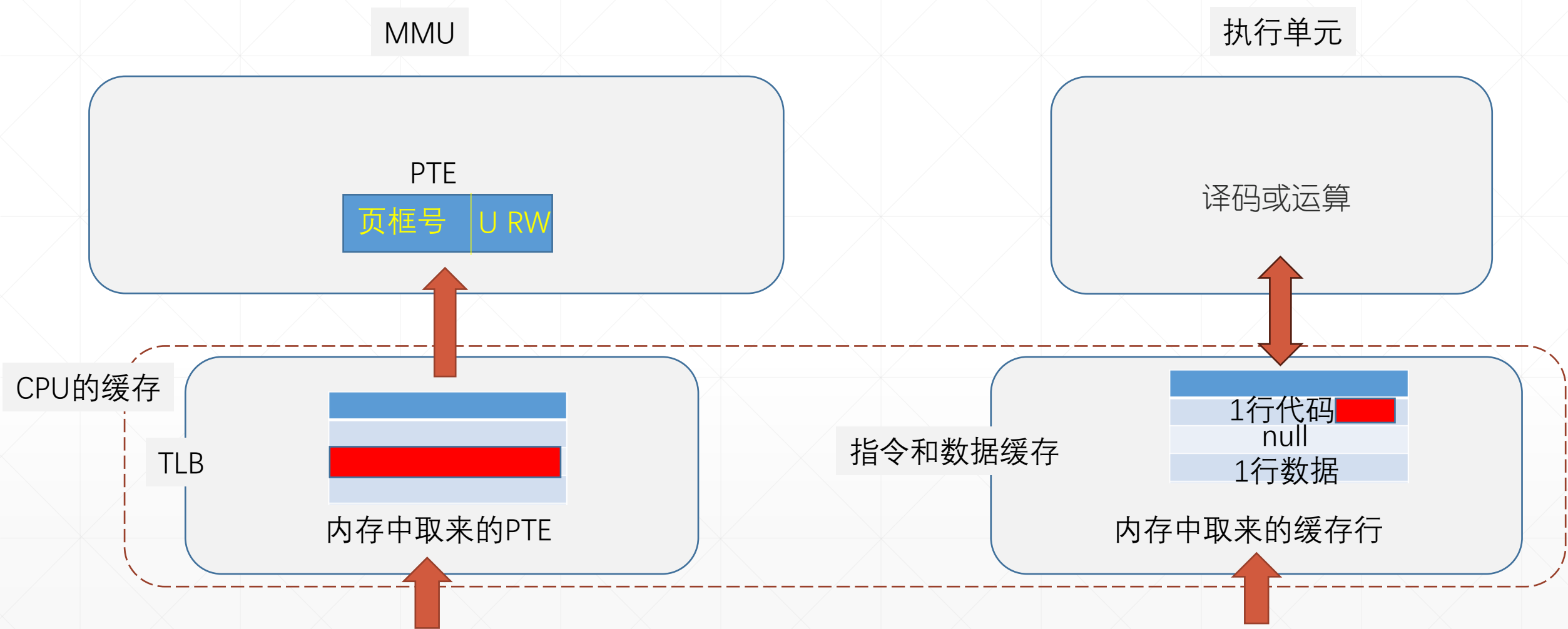


降低页表的内存开销，第二种方法：反向页表

- 设置系统页表，每个物理页框有一项。

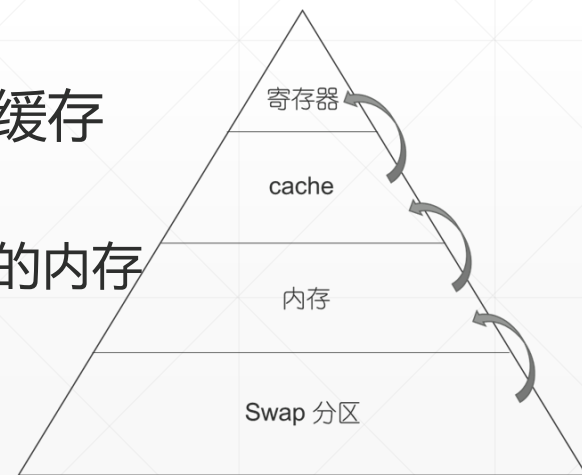


提高页式系统的访存速度，CPU设置TLB缓存最近用过的 pte



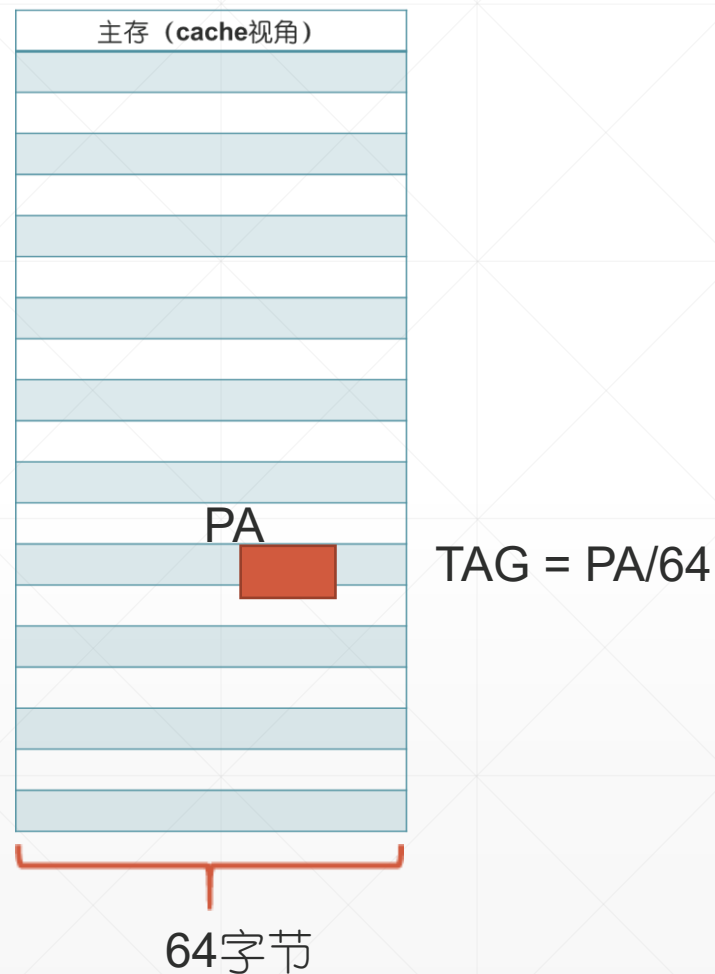
七、CPU的cache

- 作用：缓存预测中，CPU未来需要访问的指令、数据 和 PTE。减少访存耗时。
 - 指令缓存、数据缓存。用来在CPU内部缓存执行单元最近用过的指令和数据。
 - TLBs (Translation Lookaside Buffers) , 俗称快表。用来在CPU内部缓存MMU最近用过的 PTE & PDE (好像好多CPU不缓存PDE) 。
- 理想的情况：cache中存有马上要用到的数据。
- 缓存命中。硬件运行高效、经济的缓存管理算法，提高预测的准确度和 缓存命中率。
 - 置换算法。缓存不命中时，淘汰一个数据单元，用分配给它的缓存空间存放新的内存数据。常见的有2种，LRU成本高。
 - (1) 随机置换 (2) LRU (Least Recently Used) 淘汰最近最久未访问的数据



1、指令缓存和数据缓存

- 物理内存按cache line访问。
一个cache行是64字节对齐的连续内存块。
 - 0~63, 0#行
 - 64~127, 1#行
 -
- PA, 物理地址。TAG = PA/64
是内存单元的cache行号。



带缓存层的指令、数据访存（缓存命中，读）

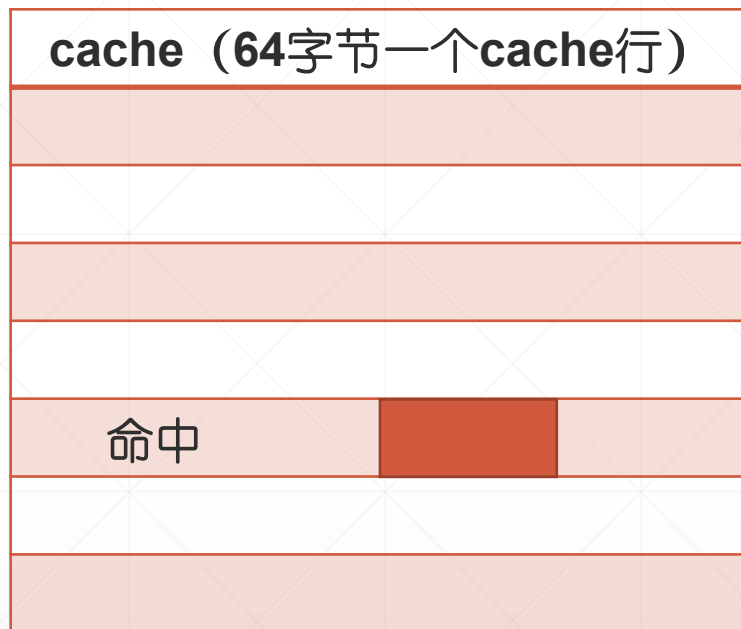
MMU给出物理地址PA访存。

2路并行

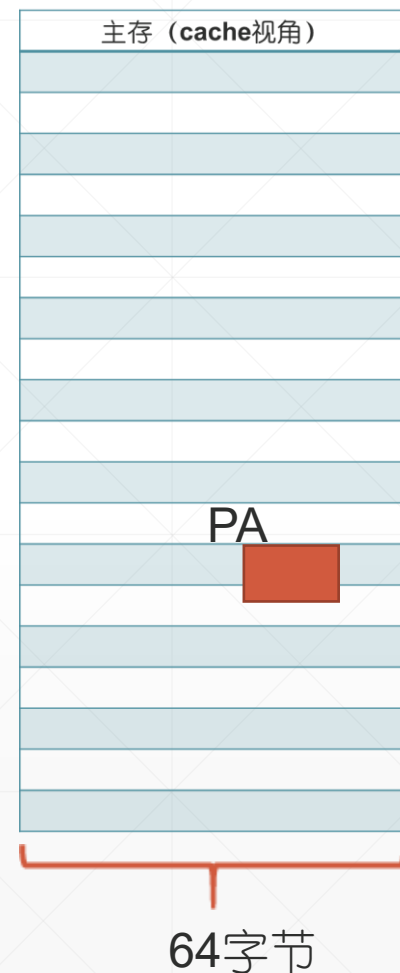
- 用TAG访问缓存。
- 用PA，通过总线访问物理内存



- 1、 $TAG = PA/64$
 $offset = PA \% 64$
- 2、用TAG查cache
- 3、缓存命中，取缓存行中偏移量是offset的数据。



耗时：0.2ns



$TAG = PA/64$

耗时：200ns

带缓存层的指令、数据访存（缓存命中，写）

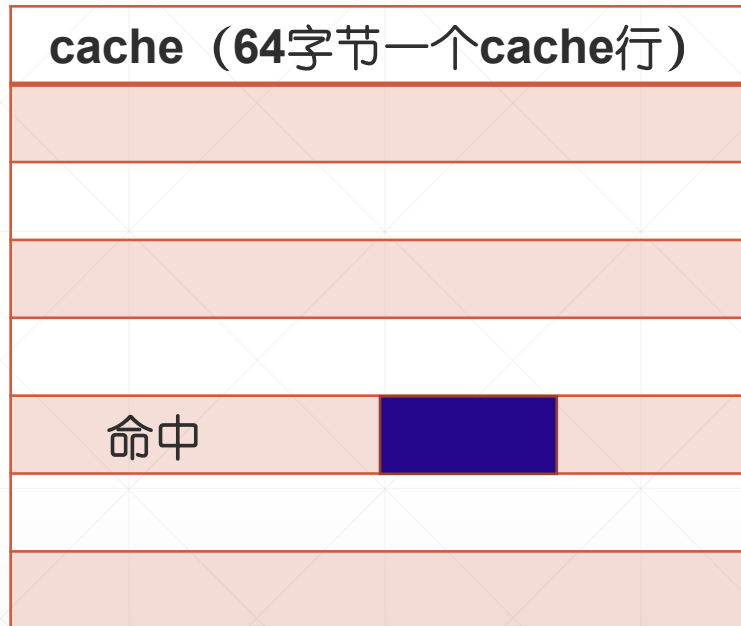
MMU给出物理地址PA访存。

2路并行

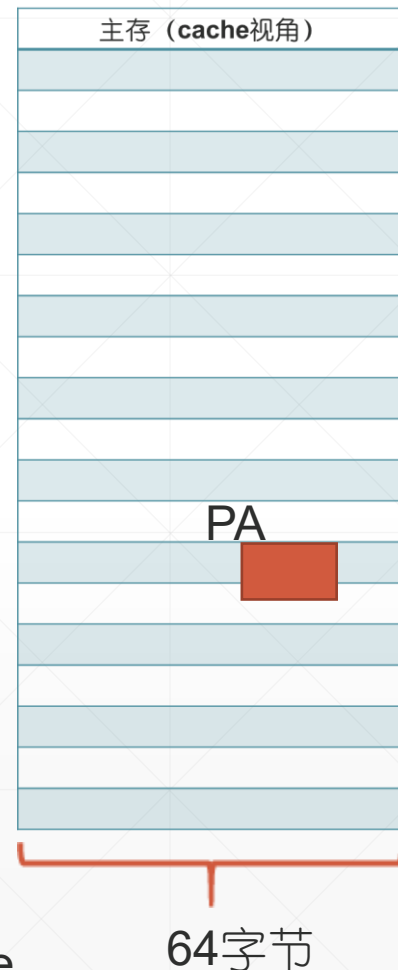
- 用TAG访问缓存。
- 用PA，通过总线访问物理内存



- 1、 $TAG = PA/64$
 $offset = PA \% 64$
- 2、用TAG查cache
- 3、缓存命中，新数据
写入缓存行中偏移量
是offset的单元。



耗时：0.2ns



$TAG = PA/64$

未来，合适的时候，新数据刷回内存。

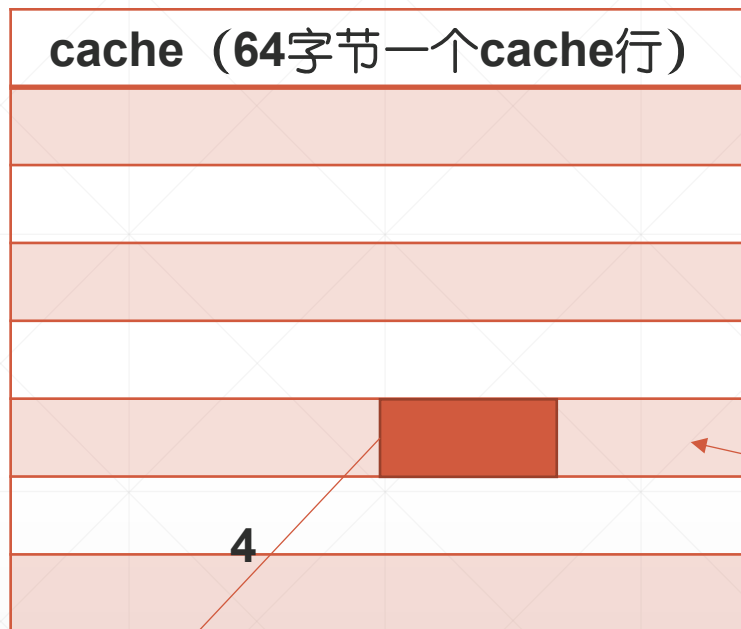
多核，物理内存中同 1 行数据有可能出现在多个CPU cache里，需要同步cache。

带缓存层的指令、数据访存（缓存不命中，读）

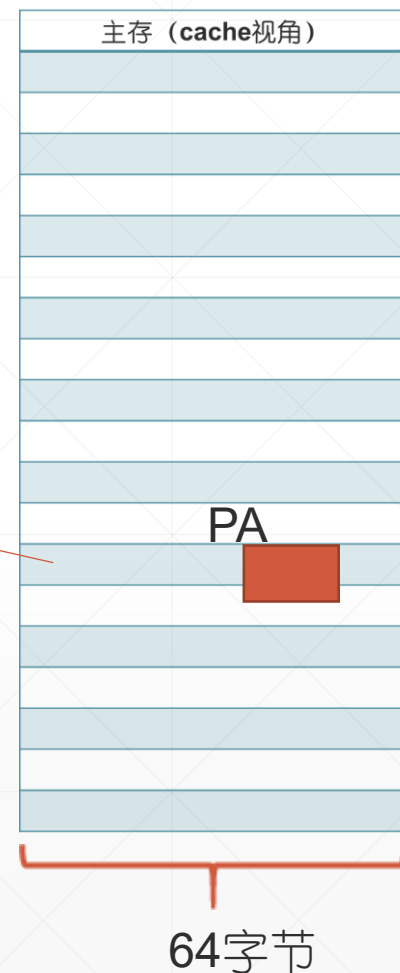
MMU给出
物理地址PA



- 1、TAG = PA/64。
offset= PA%64。
- 2、用TAG查cache。
- 3、缓存不命中，从主存中读入包含所缺数据的连续64字节，填入cache行。
- 4、取缓存行中偏移量是offset的数据。



耗时：200.2 ns



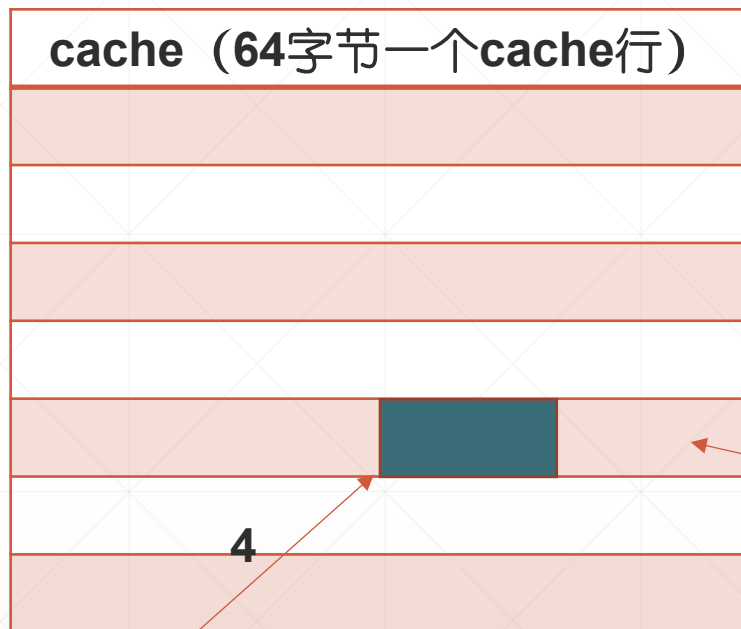
TAG = PA/64

带缓存层的指令、数据访存（缓存不命中，写）

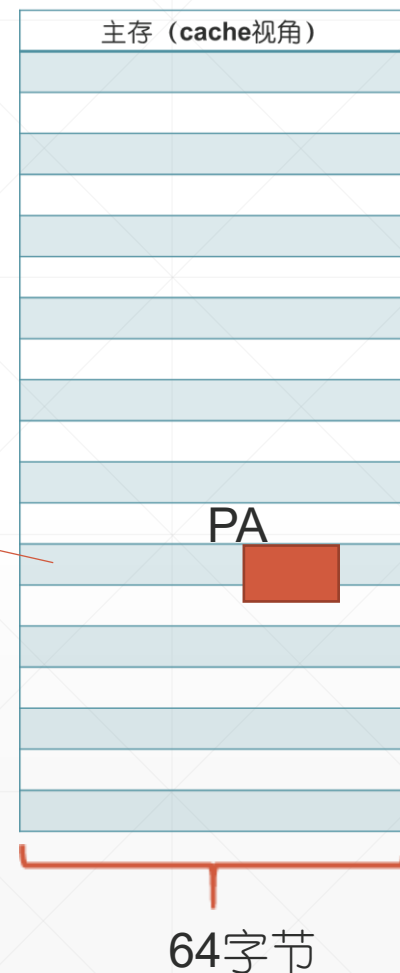
MMU给出
物理地址PA



- 1、TAG = $PA/64$ 。
offset = $PA \% 64$ 。
- 2、用TAG查cache。
- 3、先读。缓存不命中，从主存中读入包含所缺数据的连续64字节，填入cache行。
- 4、后写。新数据写入缓存行中偏移量是offset的位置。

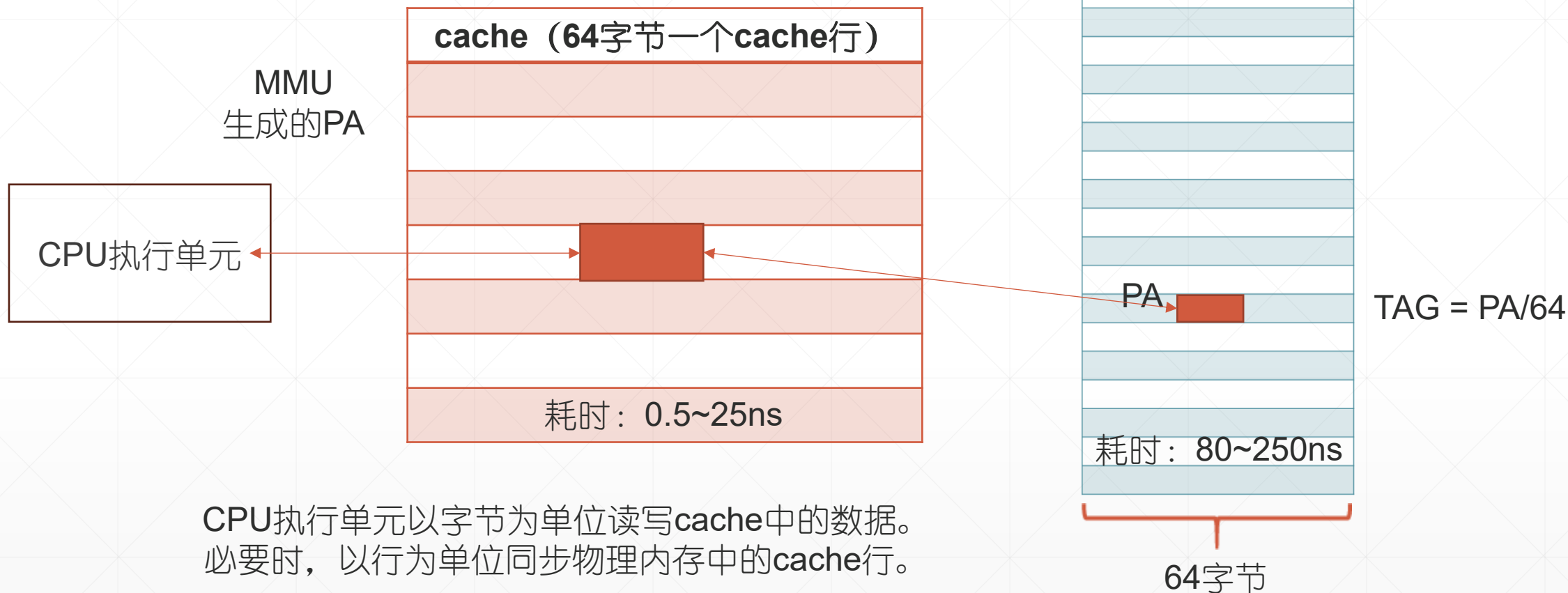


耗时：200.2ns



TAG = $PA/64$

总结：CPU使用物理地址访存的过程



CPU执行单元以字节为单位读写cache中的数据。
必要时，以行为单位同步物理内存中的cache行。

物理地址：行号TAG + offset。

TAG用来在cache和内存中定位数据所在的缓存行。offset是数据在缓存行中的偏移量。

计算：物理地址访存耗时

- 指令/数据缓存访问时间 + 缓存不命中的概率 * 主存访问时间

- 练习1:

已知缓存命中率是95%。从缓存中取数据的时间平均是10ns。访问主存单元平均耗时200ns。求该系统平均访存耗时。

$$10 + 200 * 0.05 = 20\text{ns}$$

比不设cache，主存访问速度快了10倍。

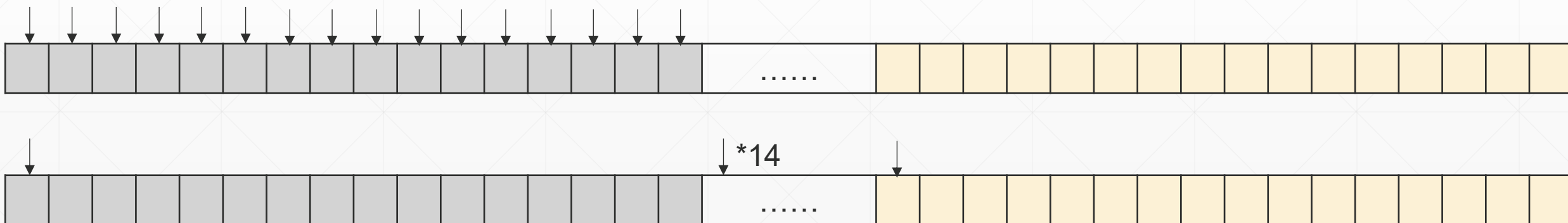
■ 习题：假设CPU只有一个缓存行。

下面2段程序，比较运行速度？解释这个现象。

height==width==16, 整数 4字节。

```
int i,j ;
sigma = 0;
for (i=0; i<height; i++)
    for (j=0; j<width; j++)
        sigma+=num[i][j];
```

```
int i,j ;
sigma = 0;
for (i=0; i<height; i++)
    for (j=0; j< width; j++)
        sigma+=num[j][i];
```



现代计算机系统，数组行主序存储。

左侧代码，每轮外层循环

- 访问num[i][0]时，CPU cache单元会从内存读入第 i 行，填入缓存。
 - 访问其它元素num[i][j]，j≠0。缓存命中。
- 这是最佳的内存访问模式：读入一个缓存行后，依次使用其中的所有数据。

右侧：每轮外层循环16次访存。先后访问16个不同的缓存行，每次用一个元素。
每次数组元素访问，均需要访存取数据。
cache没有得到利用，是最差内存访问模式。

2、TLB (装PTE的缓存)

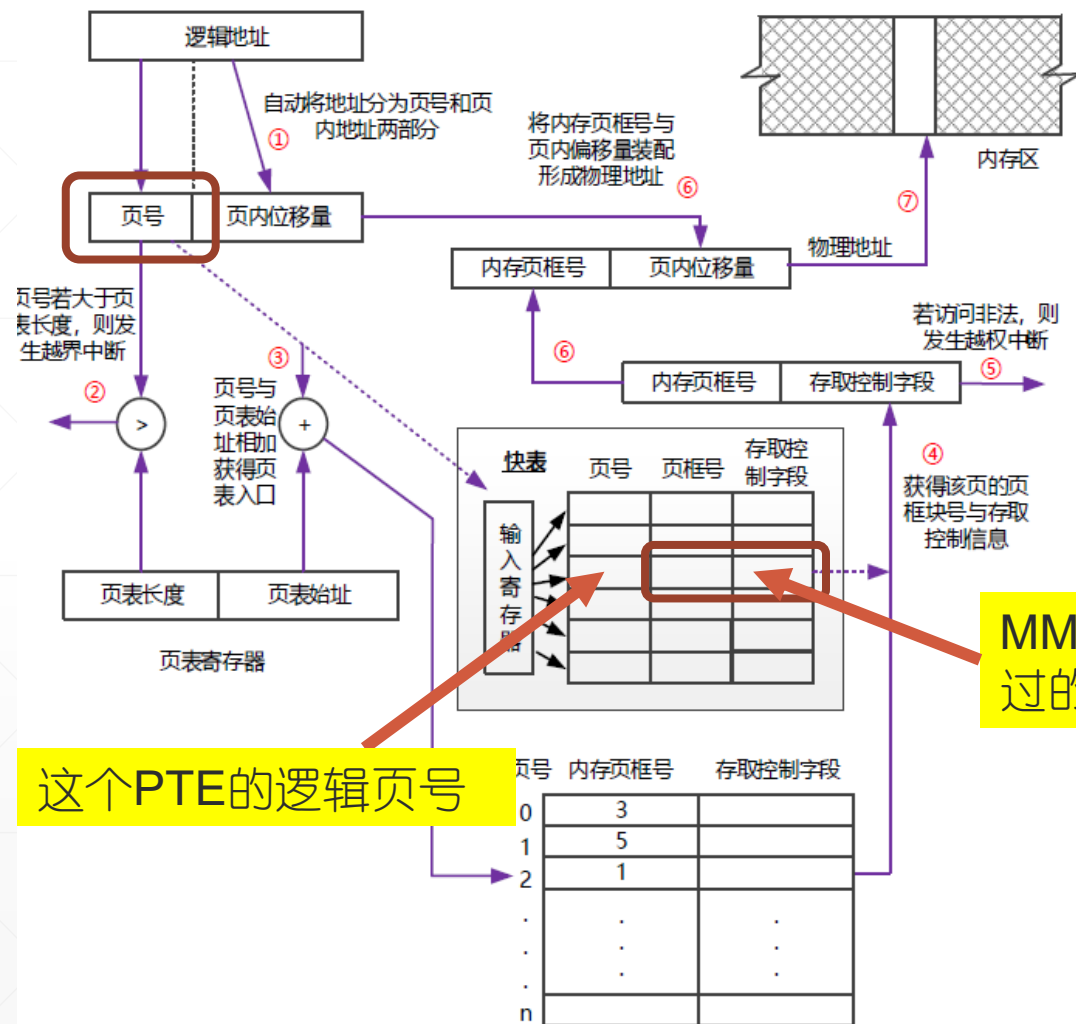
MMU在进行地址映射时，2路同时 用页号 **P** 查表：

- 一路查主存中的页表

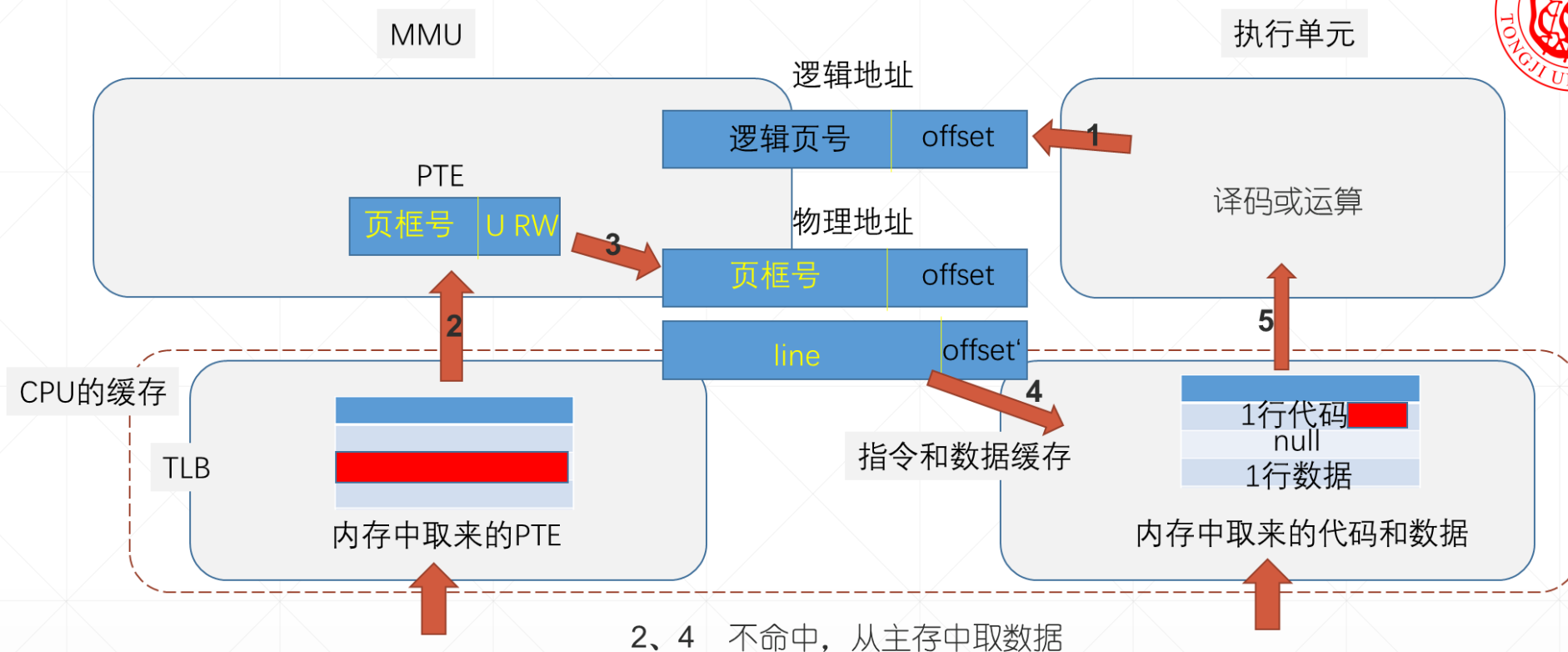
$\text{PTE地址} = \text{页表首地址} + P * \text{PTE长度}$

- 另一路查TLB

- 找到：TLB hit命中，立即形成物理地址
- 没找到，一个主存周期后，MMU会得到PTE
PTE送TLB（可能会在TLB中淘汰一个PTE），
并形成物理地址



总结：用逻辑地址访存



- 1、执行单元给出逻辑地址。MMU按逻辑页长度将其分割成 逻辑页号P 和 页内偏移量offset。
- 2、MMU，用P查TLB，读取映射所需PTE。如果不命中，需访问内存中的页表，将目标PTE存入TLB。
- 3、处理PTE。取页框号，配offset，生成物理地址。
- 4、缓存管理单元，按cache 行的长度分割物理地址，得到行号line 和 行内偏移量offset'。
- 5、用 line 查缓存 (L1/L2/L3)，看是否命中。如果不命中，访问内存取内存行。将其装入缓存。
- 6、用 offset' 访问目标缓存行，读写缓存行中的数据。



计算：EAT(Effective Access Time, 内存有效访问时间) 逻辑地址访存耗时

- $EAT = \text{地址映射时间 (访问页表时间)} + \text{访问指令/数据时间}$
 $= \text{TLB访问时间} + \text{TLB不命中的概率} * \text{主存访问页表耗时}$
 $+ \text{缓存访问时间} + \text{缓存不命中的概率} * \text{主存访问耗时}$
- 练习3：一级页表。

二级页表。PTE
没有命中TLB的
话，MMU就。。。
先去主存找PDE，
然后去主存找
PTE并缓存，最
后去主存找数据

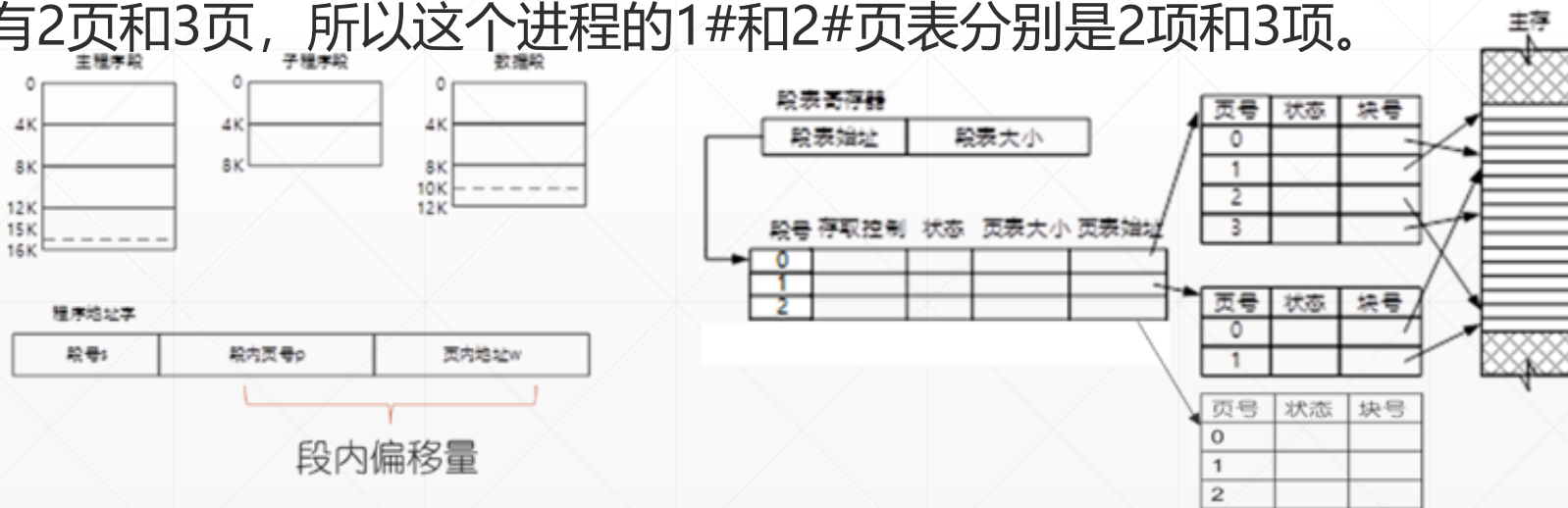
某一级页表内存管理系统，TLB 命中率是99%。从TLB中取PTE的平均耗时是10ns。指令、数据缓存命中率是95%。从缓存中取指令/数据的平均耗时是10ns。访问主存单元平均耗时200ns (取PTE、访问指令/数据相同)。求该系统内存有效访问时间。

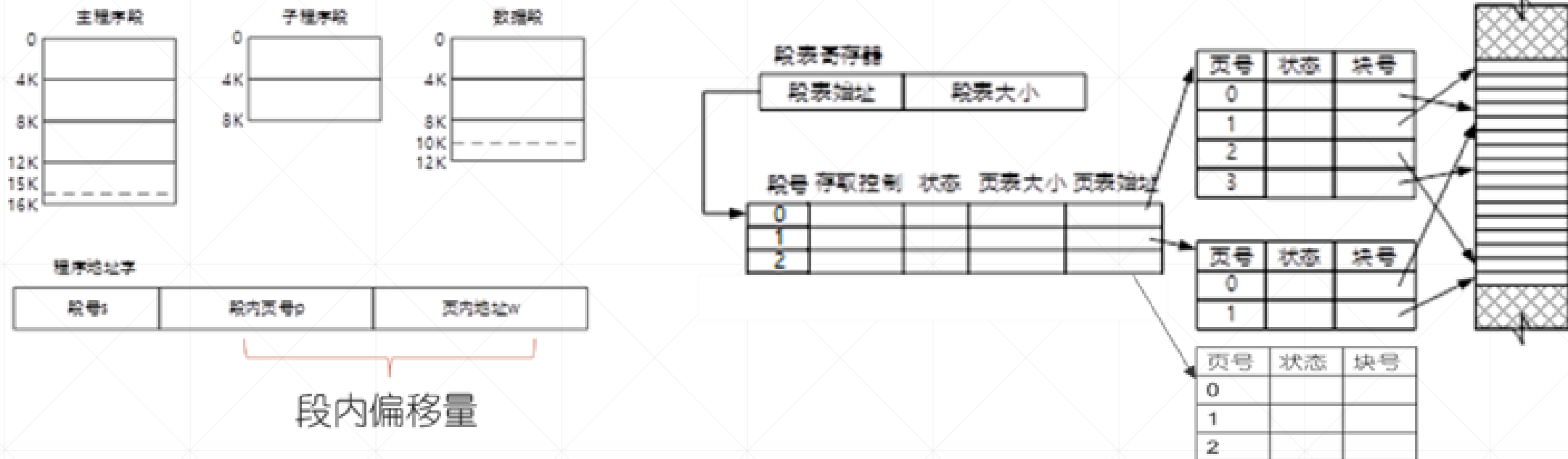
$$EAT = (10 + 200 * 0.01) + (10 + 200 * 0.05) = 32\text{ns}$$

不设cache, $EAT = 400\text{ns}$ 。使用 cache, 访存提速10倍以上。

八、段页式

- 段式内存管理方式数据共享和保护很方便，页式内存管理方式能够有效提高内存利用率和虚地址空间扩展方便。集2者的优点，段页式存储管理对虚空间分段、按页对齐。实空间分页，页式管理。
- 系统为每个进程设段表，为每个段设独立的页表。下图是一个有3段的程序，所以这个段表有3项。0#段是主程序段，对齐后有4页，所以0#页表4页。1#段、2#段分别有2页和3页，所以这个进程的1#和2#页表分别是2项和3项。





CPU内部设段表寄存器，存现运行进程段表的起始地址和长度。

段页式存储管理使用二维虚地址。段号：段内偏移量。地址转换时，MMU硬件用段号查段表。对段内偏移量进行分割，用高位页号p访问页表，得主存页框号（也就是图中得块号块号），低位拼页内地址，形成物理地址访存。所以，在段页式存储管理系统中，每次取指/操作数，需要3次访存。



九、空闲资源管理

- 连续、段式。用空闲分区表。
- 页式、段页式。用位示图。

9.1 空闲分区表

- 连续内存管理方式使用空闲分区表map 管理内存中的空闲区。
 - 空闲区未分配，已分配内存区中存有一个正文段或一个进程的可交换部分。
 - 空闲分区表管理空闲区。每个表项管理一个空闲区，登记首地址和长度。

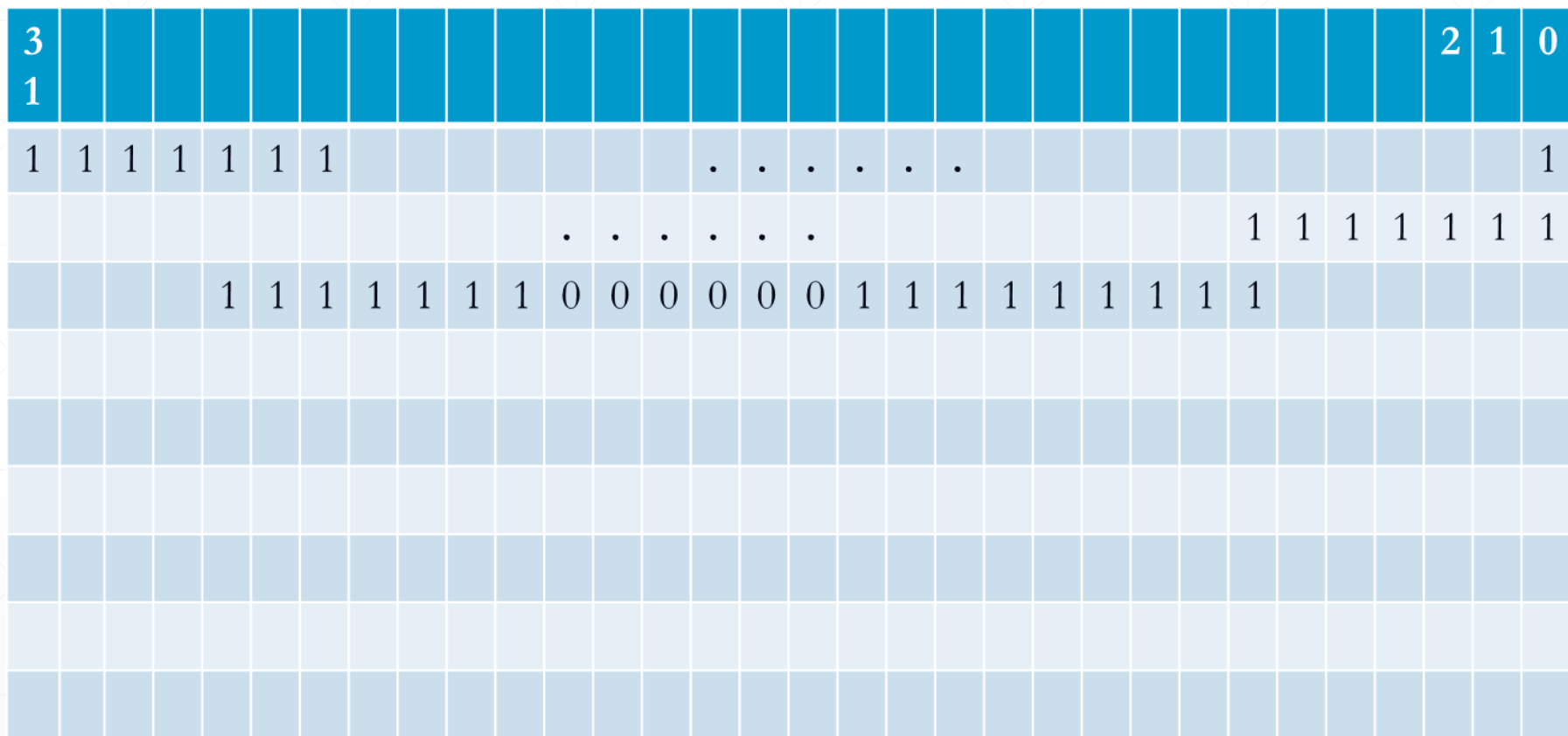
空闲分区表 map

编号	首地址	长度
1	10k	20k
2	50k	10k
3	100k	50k
4	180k	20k



9.2 位示图

- **操作系统使用位示图登记内存中物理页框的分配情况。**
内存的每个物理页框对应于一个bit
 - 0 相应的页框空闲可供使用,
 - 1 该页框中已被其它逻辑页面占用。
- **当进程需要内存空间时，操作系统扫描位示图寻找值为0的bit，将对应主存页框分配给该进程。**
- **进程运行结束或出于某种原因释放内存空间时，操作系统找到所释放的所有页框，在bit图中将它们对应的bit改为0。**



第 i 个字的第 j 个bit对应于主存页框 $32*i+j$